Dokumentacja Techniczna Systemu Bazy Danych

System ERP/MRP dla firmy produkcyjno-usługowej (Meble)

Dokumentacja Projektowa

26 stycznia 2026

Spis treści

1 Schemat bazy danych i opis tabel **5**

1.1 Tabela: categories_for_part 5
1.2 Tabela: categories_for_product 5
1.3 Tabela: clients 5
1.4 Tabela: clients_company 5
1.5 Tabela: clients_individual 6
1.6 Tabela: employee_time_offs 6
1.7 Tabela: employees 6
1.8 Tabela: inventory_parts 7
1.9 Tabela: inventory_products 7
1.10 Tabela: order_details 7
1.11 Tabela: order_parts 7
1.12 Tabela: order_status 8
1.13 Tabela: orders 8
1.14 Tabela: parts 8
1.15 Tabela: payments 9
1.16 Tabela: product_recipes 9
1.17 Tabela: production_status 9
1.18 Tabela: production_team_assignments 10
1.19 Tabela: productions 10
1.20 Tabela: products 10
1.21 Tabela: products_vat 10
1.22 Tabela: roles 11
1.23 Tabela: shippers 11
1.24 Tabela: status_for_order 11

1.25 Tabela: status_for_production . . . 11
1.26 Tabela: supplier_offers . . . 12
1.27 Tabela: suppliers . . . 12
1.28 Tabela: vat . . . 12
1.29 Tabela: work_places . . . 12

2 Zdefiniowane relacje między tabelami (SQL) 13

3 Spis widoków (Views) 16
3.1 v_inventory_parts_status (KD) . . . 16
3.2 v_supplier_offers_detail(KD) . . . 16
3.3 v_order_parts_status(KD) . . . 16
3.4 v_delivery_delays(KD) . . . 17
3.5 v_supplier_summary(KD) . . . 17
3.6 v_client_company_stats(BZ) . . . 17
3.7 v_client_individual_stats(BZ) . . . 18
3.8 v_employee_total_time_off_days(BZ) . . . 18
3.9 v_current_and_future_time_offs(BZ) . . . 18
3.10 v_client_contact_book(BZ) . . . 19
3.11 v_product_category_performance(MG) . . . 19
3.12 v_product_sales_summary(MG) . . . 20
3.13 v_product_last_order(MG) . . . 20
3.14 v_inventory_products_status(MG) . . . 20
3.15 v_product_profitability_analysis(MG) . . . 21
3.16 v_work_place_status(BZ) . . . 21
3.17 v_product_production_frequency(BZ) . . . 22
3.18 v_production_delay_details(BZ) . . . 22
3.19 v_supervisor_performance_delays(BZ) . . . 23
3.20 v_work_place_usage_stats(BZ) . . . 23
3.21 v_employee_availability_calendar(BZ) . . . 23
3.22 v_production_planned(BZ) . . . 24
3.23 v_daily_order_stats(KD) . . . 24
3.24 v_order_financial_summary(KD) . . . 25
3.25 v_active_delayed_orders(KD) . . . 25
3.26 v_orders_scheduled_today(KD) . . . 25
3.27 v_total_orders_scheduled_per_day(KD) . . . 26
3.28 v_critical_orders_due_today(KD) . . . 26
3.29 Widoki słownikowe UI . . . 26

4 Logika Biznesowa: Funkcje, Wyzwalacze i Procedury **27**
4.1 Funkcje (Functions) . . . 27
4.1.1 fn_part_offers_by_deadline(KD) . . . 27
4.1.2 fn_get_supplier_offers_by_name_or_id(KD) . . . 27
4.1.3 fn_get_late_orders_up_to_date(KD) . . . 28
4.1.4 fn_get_employee_future_time_offs(BZ) . . . 28
4.1.5 fn_get_revenue_report(MG) . . . 29
4.1.6 fn_production_cost_profit_report(MG) . . . 30
4.1.7 fn_production_costs(MG) . . . 32
4.1.8 fn_production_in_progress(BZ) . . . 32
4.1.9 fn_estimate_order_delivery(KD) . . . 32
4.1.10 fn_get_client_purchase_history(KD) . . . 33
4.1.11 fn_get_production_team(BZ) . . . 34
4.1.12 fn_get_employee_production_assignments . . . 34
4.1.13 fn_get_product_recipe_details(BZ) . . . 35
4.2 Wyzwalacze (Triggers) . . . 36
4.2.1 trg_update_stock_on_delivery(BZ) . . . 36
4.2.2 trg_auto_plan_production(KD) . . . 36
4.2.3 trg_auto_order_parts(MG) . . . 37
4.3 Procedury Składowane (Stored Procedures) . . . 38
4.3.1 sp_upsert_supplier(KD) . . . 38
4.3.2 sp_upsert_supplier_offer(KD) . . . 39
4.3.3 sp_add_new_part_type(KD) . . . 40
4.3.4 sp_place_part_order(KD) . . . 40
4.3.5 sp_adjust_part_inventory(KD) . . . 41
4.3.6 sp_add_new_client(BZ) . . . 42
4.3.7 sp_update_client_data(BZ) . . . 43
4.3.8 sp_add_employee_role(BZ) . . . 44
4.3.9 sp_add_new_employee(BZ) . . . 44
4.3.10 sp_update_employee(BZ) . . . 45
4.3.11 sp_add_employee_time_off(BZ) . . . 46
4.3.12 sp_upsert_product(MG) . . . 46
4.3.13 sp_update_product_vat(MG) . . . 48
4.3.14 sp_upsert_product_recipe(MG) . . . 49
4.3.15 sp_add_product_category(MG) . . . 49
4.3.16 sp_upsert_production(KD) . . . 50
4.3.17 sp_upsert_employee_to_production(KD) . . . 51

4.3.18 sp_remove_employee_from_production(MG) 51
4.3.19 sp_replace_production_employee(MG) 51
4.3.20 sp_start_production(KD) 52
4.3.21 sp_finish_production(MG) 54
4.3.22 sp_add_work_place(MG) 55
4.3.23 sp_confirm_order(BZ) 55
4.3.24 sp_upsert_order(BZ) 56
4.3.25 sp_add_order_detail_with_overflow(BZ) 56
4.3.26 sp_delete_order_detail(BZ) 58
4.3.27 sp_upsert_shipper(BZ) 58
4.3.28 sp_upsert_payment(BZ) 59
4.3.29 sp_manage_order_status(BZ) 59

5 Informacje odnośnie wygenerowanych danych 61
5.1 Proces generowania danych 61
5.2 Kluczowe słowniki systemowe 61

6 Logika Biznesowa: Przeplyw Zamowienia przez Magazyn 62

7 Określenie uprawnień do danych (Role) 62

8 Indeksy 63
8.1 Wykaz indeksow 63

1 Schemat bazy danych i opis tabel

W tej sekcji przedstawiono strukturę tabel, opis przechowywanych danych, zdefiniowane typy oraz warunki integralności. Baza składa się z 29 tabel powiązanych relacjami.

1.1 Tabela: categories_for_part

Słownik kategorii części (np. metalowe, drewniane).

```sql
CREATE TABLE categories_for_part (
    category_id int  NOT NULL IDENTITY(1, 1),
    name nvarchar(100)  NOT NULL,
    CONSTRAINT categories_for_part_pk PRIMARY KEY  (category_id)
);
```

Listing 1: Kod tabeli categories_for_part

1.2 Tabela: categories_for_product

Słownik kategorii produktów gotowych (np. krzesła, biurka).

```sql
CREATE TABLE categories_for_product (
    category_id int  NOT NULL IDENTITY(1, 1),
    name nvarchar(100)  NOT NULL,
    CONSTRAINT categories_for_product_pk PRIMARY KEY  (category_id)
);
```

Listing 2: Kod tabeli categories_for_product

1.3 Tabela: clients

Tabela bazowa przechowująca wspólne dane kontaktowe dla wszystkich typów klientów.

```sql
CREATE TABLE clients (
    client_id int  NOT NULL IDENTITY(1, 1),
    email nvarchar(150)  NOT NULL,
    phone nvarchar(12)  NOT NULL,
    city nvarchar(100)  NOT NULL,
    address nvarchar(150)  NOT NULL,
    city_code nvarchar(10)  NOT NULL,
    CONSTRAINT check_clients CHECK (((phone LIKE '[0-9]%' AND phone NOT LIKE '%[^0-9]%')
     OR  (phone LIKE '+[0-9]%' AND SUBSTRING(phone, 2, 12) NOT LIKE '%[^0-9]%') ) and
    email LIKE '_%@_%.%_'),
    CONSTRAINT clients_pk PRIMARY KEY  (client_id)
);
```

Listing 3: Kod tabeli clients

1.4 Tabela: clients_company

Rozszerzenie tabeli klientów przechowujące dane specyficzne dla firm (NIP, Nazwa firmy). Relacja 1:1 z tabelą `clients`.

```sql
CREATE TABLE clients_company (
    client_id int  NOT NULL,
    company_name nvarchar(255)  NOT NULL,
    nip nvarchar(10)  NOT NULL,
    CONSTRAINT clients_company_pk PRIMARY KEY  (client_id)
);
```

Listing 4: Kod tabeli clients_company

1.5 Tabela: clients_individual

Rozszerzenie tabeli klientów przechowujące dane specyficzne dla osób fizycznych (Imię, Nazwisko). Relacja 1:1 z tabelą `clients`.

```sql
CREATE TABLE clients_individual (
    client_id int  NOT NULL,
    first_name nvarchar(100)  NOT NULL,
    last_name nvarchar(100)  NOT NULL,
    CONSTRAINT clients_individual_pk PRIMARY KEY  (client_id)
);
```

Listing 5: Kod tabeli clients_individual

1.6 Tabela: employee_time_offs

Ewidencja nieobecności pracowników (urlopy, zwolnienia). Waliduje poprawność dat (koniec $\geq$ początek).

```sql
CREATE TABLE employee_time_offs (
    employee_id int  NOT NULL,
    start_date date  NOT NULL,
    end_date date  NOT NULL,
    reason nvarchar(255)  NOT NULL,
    CONSTRAINT check_employee_time_offs CHECK (end_date >= start_date),
    CONSTRAINT employee_time_offs_pk PRIMARY KEY  (employee_id,start_date)
);
```

Listing 6: Kod tabeli employee_time_offs

1.7 Tabela: employees

Główna tabela pracowników zawierająca dane kadrowe, płacowe i kontaktowe.

```sql
CREATE TABLE employees (
    employee_id int  NOT NULL IDENTITY(1, 1),
    first_name nvarchar(100)  NOT NULL,
    last_name nvarchar(100)  NOT NULL,
    base_salary money  NOT NULL,
    is_active bit  NOT NULL,
    role_id int  NOT NULL,
    supervisor_id int  NULL,
    hired_date date  NOT NULL,
    email nvarchar(150)  NOT NULL,
    phone nvarchar(20)  NOT NULL,
    CONSTRAINT check_employees CHECK (base_salary >= 0 and ( (phone LIKE '[0-9]%' AND
    phone NOT LIKE '%[^0-9]%')  OR  (phone LIKE '+[0-9]%' AND SUBSTRING(phone, 2, 12) NOT
     LIKE '%[^0-9]%') ) and email LIKE '_%@_%.%_'),
    CONSTRAINT employees_pk PRIMARY KEY  (employee_id)
```

```
);
```

Listing 7: Kod tabeli employees

1.8 Tabela: inventory_parts

Aktualny stan magazynowy części surowcowych wraz z progami minimalnymi i maksymalnymi.

```sql
CREATE TABLE inventory_parts (
    part_id int  NOT NULL,
    available_units int  NOT NULL DEFAULT 0,
    min_units int  NOT NULL,
    max_units int  NOT NULL,
    CONSTRAINT check_inventory_parts CHECK (available_units >= 0 and min_units >= 0),
    CONSTRAINT inventory_parts_pk PRIMARY KEY  (part_id)
);
```

Listing 8: Kod tabeli inventory_parts

1.9 Tabela: inventory_products

Aktualny stan magazynowy wyrobów gotowych.

```sql
CREATE TABLE inventory_products (
    product_id int  NOT NULL,
    available_units int  NOT NULL DEFAULT 0,
    min_units int  NOT NULL,
    max_units int  NOT NULL,
    CONSTRAINT check_inventory_products CHECK (available_units >= 0 and min_units >= 0),
    CONSTRAINT inventory_products_pk PRIMARY KEY  (product_id)
);
```

Listing 9: Kod tabeli inventory_products

1.10 Tabela: order_details

Szczegóły zamówienia klienta - przechowuje informacje o zamawianych produktach, ich ilości oraz cenie jednostkowej w momencie zakupu.

```sql
CREATE TABLE order_details (
    item_id int  NOT NULL IDENTITY(1, 1),
    order_id int  NOT NULL,
    product_id int  NOT NULL,
    quantity int  NOT NULL,
    unit_price money  NOT NULL,
    CONSTRAINT check_order_details CHECK (quantity > 0 and unit_price >= 0),
    CONSTRAINT order_items_pk PRIMARY KEY  (item_id)
);
```

Listing 10: Kod tabeli order_details

1.11 Tabela: order_parts

Zamówienia części składane u dostawców w celu uzupełnienia magazynu.

```sql
CREATE TABLE order_parts (
    order_part_id int  NOT NULL IDENTITY(1, 1),
    offer_id int  NOT NULL,
    order_date date  NOT NULL,
    pack_quantity int  NOT NULL,
    order_delivered date  NULL,
    CONSTRAINT check_order_parts CHECK (order_date <= order_delivered and pack_quantity > 0),
    CONSTRAINT order_parts_pk PRIMARY KEY  (order_part_id)
);
```

Listing 11: Kod tabeli order_parts

1.12 Tabela: order_status

Logi zmian statusów zamówienia klienta (historia realizacji).

```sql
CREATE TABLE order_status (
    order_status_id int  NOT NULL IDENTITY(1, 1),
    order_id int  NOT NULL,
    status_id int  NOT NULL,
    start_date datetime  NOT NULL,
    planned_end_date datetime  NOT NULL,
    end_date datetime  NULL,
    CONSTRAINT check_order_status CHECK (start_date <= planned_end_date),
    CONSTRAINT order_status_pk PRIMARY KEY  (order_status_id)
);
```

Listing 12: Kod tabeli order_status

1.13 Tabela: orders

Nagłówki zamówień klientów zawierające informacje o dostawie, rabatach i opiekunie zamówienia.

```sql
CREATE TABLE orders (
    order_id int  NOT NULL IDENTITY(1, 1),
    shipper_id int  NOT NULL,
    order_date datetime  NOT NULL DEFAULT getdate(),
    delivery_city nvarchar(100)  NOT NULL,
    delivery_address nvarchar(150)  NOT NULL,
    supervisor_id int  NOT NULL,
    order_discount decimal(3,2)  NOT NULL,
    client_id int  NOT NULL,
    delivery_city_code nvarchar(10)  NOT NULL,
    CONSTRAINT check_orders CHECK (order_discount >= 0 and order_discount <= 1),
    CONSTRAINT orders_pk PRIMARY KEY  (order_id)
);
```

Listing 13: Kod tabeli orders

1.14 Tabela: parts

Katalog części niezbędnych do produkcji.

```sql
CREATE TABLE parts (
    part_id int  NOT NULL IDENTITY(1, 1),
    part_name nvarchar(255)  NOT NULL,
```

```
    category_id int  NOT NULL,
    unit_space_needed int  NOT NULL DEFAULT 1,
    CONSTRAINT check_parts CHECK (unit_space_needed > 0),
    CONSTRAINT parts_pk PRIMARY KEY  (part_id)
);
```

Listing 14: Kod tabeli parts

1.15 Tabela: payments

Rejestr płatności powiązanych z zamówieniami.

```
CREATE TABLE payments (
    payment_id int  NOT NULL IDENTITY(1, 1),
    order_id int  NOT NULL,
    amount money  NOT NULL,
    payment_date datetime  NOT NULL,
    CONSTRAINT check_payments CHECK (amount >= 0),
    CONSTRAINT payments_pk PRIMARY KEY  (payment_id)
);
```

Listing 15: Kod tabeli payments

1.16 Tabela: product_recipes

BOM (Bill of Materials) - definicja składników (części) potrzebnych do wytworzenia produktu.

```
CREATE TABLE product_recipes (
    product_id int  NOT NULL,
    part_id int  NOT NULL,
    quantity_needed int  NOT NULL,
    CONSTRAINT check_product_recipies CHECK (quantity_needed > 0),
    CONSTRAINT product_recipes_pk PRIMARY KEY  (product_id,part_id)
);
```

Listing 16: Kod tabeli product_recipes

1.17 Tabela: production_status

Logi zmian statusów dla zleceń produkcyjnych.

```
CREATE TABLE production_status (
    production_status_id int  NOT NULL IDENTITY(1, 1),
    production_id int  NOT NULL,
    status_id int  NOT NULL,
    start_date datetime  NOT NULL,
    planned_end_date datetime  NOT NULL,
    end_date datetime  NULL,
    CONSTRAINT check_production_status CHECK (start_date <= planned_end_date),
    CONSTRAINT production_status_pk PRIMARY KEY  (production_status_id)
);
```

Listing 17: Kod tabeli production_status

1.18 Tabela: production_team_assignments

Tabela łącząca pracowników ze zleceniami produkcyjnymi, przechowująca również zaraportowany czas pracy.

```sql
CREATE TABLE production_team_assignments (
    production_id int  NOT NULL,
    employee_id int  NOT NULL,
    hours_logged int  NOT NULL,
    CONSTRAINT check_production_team_assignments CHECK (hours_logged >= 0),
    CONSTRAINT production_team_assignments_pk PRIMARY KEY  (production_id,employee_id)
);
```

Listing 18: Kod tabeli production_team_assignments

1.19 Tabela: productions

Zlecenia produkcyjne - co, ile i kto nadzoruje.

```sql
CREATE TABLE productions (
    production_id int  NOT NULL IDENTITY(1, 1),
    product_id int  NOT NULL,
    quantity int  NOT NULL,
    work_place_id int  NULL,
    supervisor_id int  NOT NULL,
    CONSTRAINT check_productions CHECK (quantity > 0),
    CONSTRAINT productions_pk PRIMARY KEY  (production_id)
);
```

Listing 19: Kod tabeli productions

1.20 Tabela: products

Katalog produktów finalnych (mebli).

```sql
CREATE TABLE products (
    product_id int  NOT NULL IDENTITY(1, 1),
    product_name nvarchar(255)  NOT NULL,
    base_price money  NOT NULL,
    unit_space_needed int  NOT NULL DEFAULT 1,
    is_active bit  NOT NULL DEFAULT 1,
    category_id int  NOT NULL,
    needed_hours int  NOT NULL,
    CONSTRAINT check_products CHECK (unit_space_needed >= 0 and needed_hours >= 0 and
    base_price >= 0),
    CONSTRAINT products_pk PRIMARY KEY  (product_id)
);
```

Listing 20: Kod tabeli products

1.21 Tabela: products_vat

Historia stawek VAT dla produktów. Pozwala na zmiany podatkowe w czasie bez utraty danych historycznych.

```sql
CREATE TABLE products_vat (
    product_id int  NOT NULL,
```

```
    vat_id int  NOT NULL,
    start_date date  NOT NULL,
    end_date date  NULL,
    CONSTRAINT check_products_vat CHECK (start_date <= end_date),
    CONSTRAINT products_vat_pk PRIMARY KEY  (product_id,vat_id,start_date)
);
```

Listing 21: Kod tabeli products_vat

1.22 Tabela: roles

Słownik stanowisk w firmie.

```
CREATE TABLE roles (
    role_id int  NOT NULL IDENTITY(1, 1),
    possition_name nvarchar(150)  NOT NULL,
    description nvarchar(255)  NOT NULL,
    CONSTRAINT roles_pk PRIMARY KEY  (role_id)
);
```

Listing 22: Kod tabeli roles

1.23 Tabela: shippers

Firmy kurierskie realizujące dostawy do klientów.

```
CREATE TABLE shippers (
    shipper_id int  NOT NULL IDENTITY(1, 1),
    company_name nvarchar(255)  NOT NULL,
    email varchar(150)  NOT NULL,
    CONSTRAINT shippers_pk PRIMARY KEY  (shipper_id)
);
```

Listing 23: Kod tabeli shippers

1.24 Tabela: status_for_order

Słownik statusów zamówień klientów.

```
CREATE TABLE status_for_order (
    status_id int  NOT NULL IDENTITY(1, 1),
    status_name nvarchar(30)  NOT NULL,
    CONSTRAINT status_for_order_pk PRIMARY KEY  (status_id)
);
```

Listing 24: Kod tabeli status_for_order

1.25 Tabela: status_for_production

Słownik statusów procesu produkcyjnego.

```
CREATE TABLE status_for_production (
    status_id int  NOT NULL IDENTITY(1, 1),
    status_name nvarchar(30)  NOT NULL,
    CONSTRAINT status_for_production_pk PRIMARY KEY  (status_id)
);
```

Listing 25: Kod tabeli status_for_production

1.26 Tabela: supplier_offers

Oferty dostawców na konkretne części (cena, czas dostawy).

```sql
CREATE TABLE supplier_offers (
    offer_id int  NOT NULL IDENTITY(1, 1),
    supplier_id int  NOT NULL,
    part_id int  NOT NULL,
    price money  NOT NULL,
    delivery_days int  NOT NULL,
    pack_size int  NOT NULL DEFAULT 1,
    CONSTRAINT check_supplier_offers CHECK (price >= 0 and delivery_days >= 0 and
    pack_size > 0),
    CONSTRAINT supplier_offers_pk PRIMARY KEY  (offer_id)
);
```

Listing 26: Kod tabeli supplier_offers

1.27 Tabela: suppliers

Dane teleadresowe dostawców.

```sql
CREATE TABLE suppliers (
    supplier_id int  NOT NULL IDENTITY(1, 1),
    company_name nvarchar(255)  NOT NULL,
    email nvarchar(150)  NOT NULL,
    CONSTRAINT check_suppliers CHECK (email LIKE '_%@_%.%_'),
    CONSTRAINT suppliers_pk PRIMARY KEY  (supplier_id)
);
```

Listing 27: Kod tabeli suppliers

1.28 Tabela: vat

Słownik stawek VAT.

```sql
CREATE TABLE vat (
    vat_id int  NOT NULL IDENTITY(1, 1),
    amount decimal(3,2)  NOT NULL,
    CONSTRAINT check_vat CHECK (amount >= 0 and amount <= 1),
    CONSTRAINT vat_pk PRIMARY KEY  (vat_id)
);
```

Listing 28: Kod tabeli vat

1.29 Tabela: work_places

Zdefiniowane stanowiska pracy (gniazda produkcyjne).

```sql
CREATE TABLE work_places (
    work_place_id int  NOT NULL IDENTITY(1, 1),
    work_place_description nvarchar(255)  NOT NULL,
    CONSTRAINT work_places_pk PRIMARY KEY  (work_place_id)
);
```

Listing 29: Kod tabeli work_places

2 Zdefiniowane relacje między tabelami (SQL)

Poniżej znajduje się kod SQL definiujący więzy integralności dla bazy danych.

```sql
-- Foreign keys

-- Reference: FK_1 (table: parts)
ALTER TABLE parts ADD CONSTRAINT FK_1
    FOREIGN KEY (category_id)
    REFERENCES categories_for_part (category_id);

-- Reference: FK_11 (table: order_details)
ALTER TABLE order_details ADD CONSTRAINT FK_11
    FOREIGN KEY (order_id)
    REFERENCES orders (order_id);

-- Reference: FK_12 (table: order_details)
ALTER TABLE order_details ADD CONSTRAINT FK_12
    FOREIGN KEY (product_id)
    REFERENCES products (product_id);

-- Reference: FK_15 (table: payments)
ALTER TABLE payments ADD CONSTRAINT FK_15
    FOREIGN KEY (order_id)
    REFERENCES orders (order_id);

-- Reference: FK_16 (table: supplier_offers)
ALTER TABLE supplier_offers ADD CONSTRAINT FK_16
    FOREIGN KEY (supplier_id)
    REFERENCES suppliers (supplier_id);

-- Reference: FK_17 (table: supplier_offers)
ALTER TABLE supplier_offers ADD CONSTRAINT FK_17
    FOREIGN KEY (part_id)
    REFERENCES parts (part_id);

-- Reference: FK_2 (table: product_recipes)
ALTER TABLE product_recipes ADD CONSTRAINT FK_2
    FOREIGN KEY (product_id)
    REFERENCES products (product_id);

-- Reference: FK_3 (table: product_recipes)
ALTER TABLE product_recipes ADD CONSTRAINT FK_3
    FOREIGN KEY (part_id)
    REFERENCES parts (part_id);

-- Reference: FK_4 (table: inventory_parts)
ALTER TABLE inventory_parts ADD CONSTRAINT FK_4
    FOREIGN KEY (part_id)
    REFERENCES parts (part_id);

-- Reference: FK_6 (table: inventory_products)
ALTER TABLE inventory_products ADD CONSTRAINT FK_6
    FOREIGN KEY (product_id)
    REFERENCES products (product_id);

-- Reference: FK_9 (table: orders)
ALTER TABLE orders ADD CONSTRAINT FK_9
    FOREIGN KEY (shipper_id)
    REFERENCES shippers (shipper_id);

-- Reference: clients_company_clients (table: clients_company)
ALTER TABLE clients_company ADD CONSTRAINT clients_company_clients
```

```sql
    FOREIGN KEY (client_id)
    REFERENCES clients (client_id);

-- Reference: clients_individual_clients (table: clients_individual)
ALTER TABLE clients_individual ADD CONSTRAINT clients_individual_clients
    FOREIGN KEY (client_id)
    REFERENCES clients (client_id);

-- Reference: employee_time_off_employees (table: employee_time_offs)
ALTER TABLE employee_time_offs ADD CONSTRAINT employee_time_off_employees
    FOREIGN KEY (employee_id)
    REFERENCES employees (employee_id);

-- Reference: employees_employees (table: employees)
ALTER TABLE employees ADD CONSTRAINT employees_employees
    FOREIGN KEY (supervisor_id)
    REFERENCES employees (employee_id);

-- Reference: employees_roles (table: employees)
ALTER TABLE employees ADD CONSTRAINT employees_roles
    FOREIGN KEY (role_id)
    REFERENCES roles (role_id);

-- Reference: order_parts_supplier_offers (table: order_parts)
ALTER TABLE order_parts ADD CONSTRAINT order_parts_supplier_offers
    FOREIGN KEY (offer_id)
    REFERENCES supplier_offers (offer_id);

-- Reference: order_status_orders (table: order_status)
ALTER TABLE order_status ADD CONSTRAINT order_status_orders
    FOREIGN KEY (order_id)
    REFERENCES orders (order_id);

-- Reference: order_status_status (table: order_status)
ALTER TABLE order_status ADD CONSTRAINT order_status_status
    FOREIGN KEY (status_id)
    REFERENCES status_for_order (status_id);

-- Reference: orders_clients (table: orders)
ALTER TABLE orders ADD CONSTRAINT orders_clients
    FOREIGN KEY (client_id)
    REFERENCES clients (client_id);

-- Reference: orders_employees (table: orders)
ALTER TABLE orders ADD CONSTRAINT orders_employees
    FOREIGN KEY (supervisor_id)
    REFERENCES employees (employee_id);

-- Reference: production_employees (table: productions)
ALTER TABLE productions ADD CONSTRAINT production_employees
    FOREIGN KEY (supervisor_id)
    REFERENCES employees (employee_id);

-- Reference: production_production_status (table: production_status)
ALTER TABLE production_status ADD CONSTRAINT production_production_status
    FOREIGN KEY (production_id)
    REFERENCES productions (production_id);

-- Reference: production_products (table: productions)
ALTER TABLE productions ADD CONSTRAINT production_products
    FOREIGN KEY (product_id)
    REFERENCES products (product_id);

```

```sql
-- Reference: production_status_production_status (table: production_status)
ALTER TABLE production_status ADD CONSTRAINT production_status_production_status
    FOREIGN KEY (status_id)
    REFERENCES status_for_production (status_id);

-- Reference: production_team_assignments_employees (table: production_team_assignments)
ALTER TABLE production_team_assignments ADD CONSTRAINT
    production_team_assignments_employees
    FOREIGN KEY (employee_id)
    REFERENCES employees (employee_id);

-- Reference: production_team_assignments_production (table: production_team_assignments)
ALTER TABLE production_team_assignments ADD CONSTRAINT
    production_team_assignments_production
    FOREIGN KEY (production_id)
    REFERENCES productions (production_id);

-- Reference: production_work_places (table: productions)
ALTER TABLE productions ADD CONSTRAINT production_work_places
    FOREIGN KEY (work_place_id)
    REFERENCES work_places (work_place_id);

-- Reference: products_product_categories (table: products)
ALTER TABLE products ADD CONSTRAINT products_product_categories
    FOREIGN KEY (category_id)
    REFERENCES categories_for_product (category_id);

-- Reference: products_vat_products (table: products_vat)
ALTER TABLE products_vat ADD CONSTRAINT products_vat_products
    FOREIGN KEY (product_id)
    REFERENCES products (product_id);

-- Reference: products_vat_vat (table: products_vat)
ALTER TABLE products_vat ADD CONSTRAINT products_vat_vat
    FOREIGN KEY (vat_id)
    REFERENCES vat (vat_id);
```

Listing 30: Skrypt SQL tworzący klucze obce

3 Spis widoków (Views)

Poniżej przedstawiono zaimplementowane widoki wraz z kodem SQL i opisem ich funkcjonalności. Są one kluczowe dla warstwy raportowej i analitycznej systemu. Zdefiniowano 28 widoków.

3.1 v_inventory_parts_status

Monitoruje stan magazynowy części. Dodaje kolumnę z komunikatem ostrzegawczym ("UWAGA: Poniżej minimum!"), jeśli stan spadnie poniżej progu.

```sql
CREATE OR ALTER VIEW v_inventory_parts_status AS
SELECT
    ip.part_id,
    p.part_name,
    ip.available_units,
    ip.min_units,
    ip.max_units,
    CASE
        WHEN ip.available_units < ip.min_units THEN 'UWAGA: Poniżej minimum!'
        ELSE 'Stan OK'
    END AS status_uwagi
FROM inventory_parts ip
JOIN parts p ON ip.part_id = p.part_id;
```

3.2 v_supplier_offers_detail

Wyświetla czytelne szczegóły ofert dostawców, łącząc identyfikatory z nazwami firm i części.

```sql
CREATE OR ALTER VIEW v_supplier_offers_detail AS
SELECT
    so.offer_id,
    p.part_name,
    s.company_name AS supplier_name,
    ROUND(so.price, 2) AS price,
    so.delivery_days,
    so.pack_size
FROM supplier_offers so
JOIN suppliers s ON so.supplier_id = s.supplier_id
JOIN parts p ON so.part_id = p.part_id;
```

3.3 v_order_parts_status

Śledzi status zamówień na części od dostawców. Oblicza planowaną datę dostawy na podstawie daty zamówienia i czasu realizacji z oferty.

```sql
CREATE OR ALTER VIEW v_order_parts_status AS
SELECT
    op.order_part_id,
    p.part_name,
    op.pack_quantity,
    so.pack_size as 'parts_in_pack',
    CASE
        WHEN op.order_delivered IS NULL THEN 'Nie dostarczone'
        ELSE 'Dostarczone'
    END AS status_dostawy,
    DATEADD(day, so.delivery_days, op.order_date) AS planned_delivery_date,
```

```
    op.order_delivered AS actual_delivery_date
FROM order_parts op
JOIN supplier_offers so ON op.offer_id = so.offer_id
JOIN parts p ON so.part_id = p.part_id;
```

3.4 v_delivery_delays

Raportuje opóźnienia w dostawach części. Wylicza dni zwłoki dla zamówień niedostarczonych w terminie lub dostarczonych po czasie.

```
CREATE OR ALTER VIEW v_delivery_delays AS
SELECT
    op.order_part_id,
    p.part_name,
    s.company_name AS supplier_name,
    DATEADD(day, so.delivery_days, op.order_date) AS planned_delivery_date,
    op.order_delivered AS actual_delivery_date,
    DATEDIFF(day, DATEADD(day, so.delivery_days, op.order_date), ISNULL(op.
    order_delivered, GETDATE())) AS delay_days,
    CASE
        WHEN op.order_delivered IS NULL THEN 'Zaległe (nie dostarczone)'
        ELSE 'Dostarczone po terminie'
    END AS delay_type
FROM order_parts op
JOIN supplier_offers so ON op.offer_id = so.offer_id
JOIN suppliers s ON so.supplier_id = s.supplier_id
JOIN parts p ON so.part_id = p.part_id
WHERE (op.order_delivered > DATEADD(day, so.delivery_days, op.order_date))
   OR (op.order_delivered IS NULL AND DATEADD(day, so.delivery_days, op.order_date) <
    GETDATE());
```

3.5 v_supplier_summary

Podsumowanie efektywności dostawców: łączna wartość zamówień, średnie opóźnienie w dniach oraz liczba opóźnionych zamówień.

```
CREATE OR ALTER VIEW v_supplier_summary AS
SELECT
    s.company_name,
    ROUND(SUM(so.price * op.pack_quantity), 2) AS total_value,
    AVG(CAST(DATEDIFF(day, DATEADD(day, so.delivery_days, op.order_date), ISNULL(op.
    order_delivered, GETDATE())) AS FLOAT)) AS avg_delay,
    COUNT(op.order_part_id) AS total_orders,
    SUM(CASE WHEN ISNULL(op.order_delivered, GETDATE()) > DATEADD(day, so.delivery_days,
    op.order_date) THEN 1 ELSE 0 END) AS delayed_orders_count
FROM suppliers s
JOIN supplier_offers so ON s.supplier_id = so.supplier_id
JOIN order_parts op ON so.offer_id = op.offer_id
GROUP BY s.supplier_id, s.company_name;
```

3.6 v_client_company_stats

Statystyki dla klientów firmowych (LTV - Lifetime Value, liczba zamówień, średnia wartość koszyka).

```
CREATE OR ALTER VIEW v_client_company_stats AS
SELECT
```

```sql
    cc.company_name,
    cc.nip,
    COUNT(o.order_id) AS total_orders_count,
    ROUND(SUM(od.quantity * od.unit_price * (1 - o.order_discount)), 2) AS total_orders_value,
    ROUND(AVG(od.quantity * od.unit_price * (1 - o.order_discount)), 2) AS average_order_value,
    MAX(o.order_date) AS last_order_date
FROM clients_company cc
JOIN orders o ON cc.client_id = o.client_id
JOIN order_details od ON o.order_id = od.order_id
GROUP BY cc.client_id, cc.company_name, cc.nip;
```

3.7 v_client_individual_stats

Analogiczne statystyki (LTV, liczba zamówień) dla klientów indywidualnych.

```sql
CREATE OR ALTER VIEW v_client_individual_stats AS
SELECT
    ci.first_name,
    ci.last_name,
    COUNT(o.order_id) AS total_orders_count,
    ROUND(SUM(od.quantity * od.unit_price * (1 - o.order_discount)), 2) AS total_orders_value,
    ROUND(AVG(od.quantity * od.unit_price * (1 - o.order_discount)), 2) AS average_order_value,
    MAX(o.order_date) AS last_order_date
FROM clients_individual ci
JOIN orders o ON ci.client_id = o.client_id
JOIN order_details od ON o.order_id = od.order_id
GROUP BY ci.client_id, ci.first_name, ci.last_name;
```

3.8 v_employee_total_time_off_days

Raport sumujący całkowitą liczbę dni wolnych wykorzystanych przez poszczególnych pracowników.

```sql
CREATE OR ALTER VIEW v_employee_total_time_off_days AS
SELECT
    e.employee_id,
    e.first_name,
    e.last_name,
    SUM(DATEDIFF(day, eto.start_date, eto.end_date) + 1) AS total_days_off
FROM employees e
JOIN employee_time_offs eto ON e.employee_id = eto.employee_id
GROUP BY e.employee_id, e.first_name, e.last_name;
```

3.9 v_current_and_future_time_offs

Lista trwających i planowanych urlopów pracowników (data końca w przyszłości).

```sql
CREATE OR ALTER VIEW v_current_and_future_time_offs AS
SELECT
    e.first_name,
    e.last_name,
    eto.start_date,
    eto.end_date,
```

```sql
    DATEDIFF(day, eto.start_date, eto.end_date) + 1 AS duration_days,
    eto.reason
FROM employee_time_offs eto
JOIN employees e ON eto.employee_id = e.employee_id
WHERE eto.end_date >= CAST(GETDATE() AS DATE);
```

3.10 v_client_contact_book

Zunifikowana książka kontaktowa łącząca klientów firmowych i indywidualnych w jeden widok.

```sql
CREATE OR ALTER VIEW v_client_contact_book AS
SELECT
    c.client_id,
    CASE
        WHEN cc.company_name IS NOT NULL THEN cc.company_name
        ELSE ci.first_name + ' ' + ci.last_name
    END AS client_display_name,
    c.email,
    c.phone,
    c.city,
    c.address,
    CASE WHEN cc.company_name IS NOT NULL THEN 'Firma' ELSE 'Indywidualny' END AS
    client_type
FROM clients c
LEFT JOIN clients_company cc ON c.client_id = cc.client_id
LEFT JOIN clients_individual ci ON c.client_id = ci.client_id;
```

3.11 v_product_category_performance

Analiza rentowności kategorii produktów. Oblicza przychód ze sprzedaży oraz szacowany koszt części (na podstawie średnich cen zakupu i receptur BOM).

```sql
CREATE OR ALTER VIEW v_product_category_performance AS
WITH PartPrices AS (
    SELECT
        part_id,
        AVG(price) AS avg_part_price
    FROM supplier_offers
    GROUP BY part_id
),
CategoryCosts AS (
    SELECT
        p.category_id,
        od.order_id,
        (od.quantity * pr.quantity_needed * pp.avg_part_price) AS line_part_cost,
        (od.quantity * od.unit_price * (1 - o.order_discount)) AS line_revenue
    FROM products p
    JOIN order_details od ON p.product_id = od.product_id
    JOIN orders o ON od.order_id = o.order_id
    JOIN product_recipes pr ON p.product_id = pr.product_id
    JOIN PartPrices pp ON pr.part_id = pp.part_id
)
SELECT
    cp.category_id,
    cp.name AS category_name,
    ROUND(SUM(cc.line_revenue), 2) AS total_sales_value,
    (SELECT SUM(od2.quantity * pr2.quantity_needed)
     FROM order_details od2
     JOIN products p2 ON od2.product_id = p2.product_id
```

```sql
     JOIN product_recipes pr2 ON p2.product_id = pr2.product_id
     WHERE p2.category_id = cp.category_id) AS total_parts_consumed,
    ROUND(SUM(cc.line_part_cost), 2) AS total_estimated_parts_cost,
    ROUND(SUM(cc.line_revenue) - SUM(cc.line_part_cost), 2) AS net_income_after_parts
FROM categories_for_product cp
LEFT JOIN CategoryCosts cc ON cp.category_id = cc.category_id
GROUP BY cp.category_id, cp.name;
```

3.12 v_product_sales_summary

Podsumowanie wolumenu i wartości sprzedaży dla poszczególnych produktów.

```sql
CREATE OR ALTER VIEW v_product_sales_summary AS
SELECT
    p.product_id,
    p.product_name,
    SUM(od.quantity) AS total_units_sold,
    ROUND(SUM(od.quantity * od.unit_price * (1 - o.order_discount)), 2) AS
    total_sales_value
FROM products p
LEFT JOIN order_details od ON p.product_id = od.product_id
LEFT JOIN orders o ON od.order_id = o.order_id
GROUP BY p.product_id, p.product_name;
```

3.13 v_product_last_order

Informacja o dacie ostatniego zamówienia dla każdego produktu (analiza rotacji towaru).

```sql
CREATE OR ALTER VIEW v_product_last_order AS
SELECT
    p.product_id,
    p.product_name,
    MAX(o.order_date) AS last_order_date
FROM products p
JOIN order_details od ON p.product_id = od.product_id
JOIN orders o ON od.order_id = o.order_id
GROUP BY p.product_id, p.product_name;
```

3.14 v_inventory_products_status

Monitorowanie stanów magazynowych produktów gotowych. Generuje alerty przy stanach zerowych lub poniżej minimum.

```sql
CREATE OR ALTER VIEW v_inventory_products_status AS
SELECT
    ip.product_id,
    p.product_name,
    ip.available_units,
    ip.min_units,
    ip.max_units,
    CASE
        WHEN ip.available_units < ip.min_units THEN 'UWAGA: Poniżej minimum!'
        WHEN ip.available_units = 0 THEN 'BRAK NA STANIE'
        ELSE 'Stan OK'
    END AS stock_alert
FROM inventory_products ip
JOIN products p ON ip.product_id = p.product_id;
```

3.15 v_product_profitability_analysis

Analiza marży brutto na jednostkę produktu (Cena bazowa minus koszt materiałowy).

```sql
CREATE OR ALTER VIEW v_product_profitability_analysis AS
WITH AvgPartPrices AS (
    SELECT
        part_id,
        AVG(price) AS avg_price
    FROM supplier_offers
    GROUP BY part_id
),
ProductPartsCosts AS (
    SELECT
        pr.product_id,
        SUM(pr.quantity_needed * ISNULL(ap.avg_price, 0)) AS total_parts_cost
    FROM product_recipes pr
    LEFT JOIN AvgPartPrices ap ON pr.part_id = ap.part_id
    GROUP BY pr.product_id
)
SELECT
    p.product_id,
    p.product_name,
    ROUND(p.base_price, 2) AS product_sale_price,
    ROUND(ISNULL(ppc.total_parts_cost, 0), 2) AS total_parts_cost,
    ROUND(p.base_price - ISNULL(ppc.total_parts_cost, 0), 2) AS gross_profit_margin,
    CASE
        WHEN p.base_price > 0 THEN
            ROUND(((p.base_price - ISNULL(ppc.total_parts_cost, 0)) / p.base_price) *
    100, 2)
        ELSE 0
    END AS margin_percentage
FROM products p
LEFT JOIN ProductPartsCosts ppc ON p.product_id = ppc.product_id;
```

3.16 v_work_place_status

Dashboard dla hali produkcyjnej. Pokazuje status stanowisk, co jest na nich produkowane, kto nadzoruje proces i kiedy planowany jest koniec.

```sql
CREATE OR ALTER VIEW v_work_place_status AS
WITH CurrentProduction AS (
    SELECT
        p.work_place_id,
        p.production_id,
        pr.product_name,
        ps.start_date,
        ps.planned_end_date,
        p.supervisor_id,
        e.first_name + ' ' + e.last_name AS supervisor_name
    FROM productions p
    JOIN products pr ON p.product_id = pr.product_id
    JOIN production_status ps ON p.production_id = ps.production_id
    JOIN Employees as e on e.employee_id=p.supervisor_id
    WHERE ps.status_id = 2        -- 'In Production'
      AND ps.end_date IS NULL     -- Jeszcze nie zakonczona
)
SELECT
    wp.work_place_id,
    wp.work_place_description,
    cp.production_id,
```

```sql
    cp.product_name AS current_product,
    cp.start_date,
    cp.planned_end_date AS estimated_free_date,
    cp.supervisor_id,
    cp.supervisor_name,
    CASE
        WHEN cp.production_id IS NOT NULL THEN 'Zajęte'
        ELSE 'Wolne'
    END AS workplace_status
FROM work_places wp
LEFT JOIN CurrentProduction cp ON wp.work_place_id = cp.work_place_id;
```

3.17 v_product_production_frequency

Ranking najczęściej produkowanych wyrobów (według liczby zleceń).

```sql
CREATE OR ALTER VIEW v_product_production_frequency AS
SELECT
    p.product_id,
    pr.product_name,
    COUNT(p.production_id) AS production_orders_count,
    SUM(p.quantity) AS total_quantity_produced
FROM productions p
JOIN products pr ON p.product_id = pr.product_id
GROUP BY p.product_id, pr.product_name;
```

3.18 v_production_delay_details

Raport opóźnień produkcyjnych. Porównuje planowaną datę zakończenia z faktyczną (lub bieżącą, jeśli proces trwa).

```sql
CREATE OR ALTER VIEW v_production_delay_details AS
SELECT
    p.production_id,
    pr.product_name,
    p.quantity,
    ps.start_date,
    ps.planned_end_date,
    ps.end_date AS actual_end_date,
    CASE
        WHEN ps.end_date IS NOT NULL AND ps.end_date > ps.planned_end_date
             THEN DATEDIFF(day, ps.planned_end_date, ps.end_date)
        WHEN ps.end_date IS NULL AND GETDATE() > ps.planned_end_date
             THEN DATEDIFF(day, ps.planned_end_date, GETDATE())
        ELSE 0
    END AS delay_days,
    p.supervisor_id,
    p.work_place_id,
    CASE
        WHEN (ps.end_date IS NOT NULL AND ps.end_date > ps.planned_end_date)
             OR (ps.end_date IS NULL AND GETDATE() > ps.planned_end_date) THEN '
    Opóźnienie'
        WHEN ps.end_date IS NULL AND ps.status_id = 2 THEN 'W produkcji'
        ELSE 'W terminie / Planowo'
    END AS status_delay
FROM productions p
JOIN products pr ON p.product_id = pr.product_id
JOIN production_status ps ON p.production_id = ps.production_id;
```

3.19 v_supervisor_performance_delays

Ocena pracy kierowników produkcji pod kątem liczby nadzorowanych opóźnionych zleceń.

```sql
CREATE OR ALTER VIEW v_supervisor_performance_delays AS
SELECT
    e.employee_id,
    e.first_name + ' ' + e.last_name AS supervisor_name,
    COUNT(p.production_id) AS total_supervised,
    SUM(CASE
        WHEN (ps.end_date IS NOT NULL AND ps.end_date > ps.planned_end_date)
             OR (ps.end_date IS NULL AND GETDATE() > ps.planned_end_date) THEN 1
        ELSE 0
    END) AS count_of_delayed_productions
FROM employees e
JOIN productions p ON e.employee_id = p.supervisor_id
JOIN production_status ps ON p.production_id = ps.production_id
GROUP BY e.employee_id, e.first_name, e.last_name;
```

3.20 v_work_place_usage_stats

Statystyka wykorzystania stanowisk produkcyjnych (ilość obsłużonych zleceń).

```sql
CREATE OR ALTER VIEW v_work_place_usage_stats AS
SELECT
    wp.work_place_id,
    wp.work_place_description,
    COUNT(p.production_id) AS total_productions_handled
FROM work_places wp
LEFT JOIN productions p ON wp.work_place_id = p.work_place_id
GROUP BY wp.work_place_id, wp.work_place_description;
```

3.21 v_employee_availability_calendar

Kalendarz dostępności pracowników produkcyjnych. Uwzględnia aktualne przypisania do zleceń oraz planowane urlopy.

```sql
CREATE OR ALTER VIEW v_employee_availability_calendar AS
WITH CurrentWork AS (
 -- jezeli pracownik pracuje, to kiedy konczy?
    SELECT
        pta.employee_id,
        MAX(ps.planned_end_date) AS busy_until
    FROM production_team_assignments pta
    JOIN production_status ps ON pta.production_id = ps.production_id
    WHERE ps.status_id = 2 AND ps.end_date IS NULL
    GROUP BY pta.employee_id
),
NextTimeOff AS (
    -- najblizszy urlop
    SELECT
        employee_id,
        MIN(start_date) AS next_off_start
    FROM employee_time_offs
    WHERE start_date >= GETDATE()
    GROUP BY employee_id
)
SELECT
    e.employee_id AS [Employee ID],
```

```sql
    e.first_name + ' ' + e.last_name AS [Full Name],
    e.email,
    r.possition_name AS [Role],
    cw.busy_until AS [Available From],
    nto.next_off_start AS [Free Until (Next Time Off)],
    CASE
        WHEN cw.busy_until IS NOT NULL THEN 'Pracuje'
        ELSE 'Dostępny teraz'
    END AS [Current State]
FROM employees e
JOIN roles r ON e.role_id = r.role_id
LEFT JOIN CurrentWork cw ON e.employee_id = cw.employee_id
LEFT JOIN NextTimeOff nto ON e.employee_id = nto.employee_id
WHERE e.is_active = 1
  AND e.role_id BETWEEN 1 AND 3;
```

3.22 v_production_planned

Widok pomocniczy wyświetlający tylko zaplanowane (jeszcze nie rozpoczęte) zlecenia produkcyjne.

```sql
CREATE OR ALTER VIEW v_production_planned AS
SELECT production_id,product_name,quantity,supervisor_id,start_date,planned_end_date FROM v_production_delay_details
WHERE actual_end_date IS NULL AND work_place_id IS NULL;
```

3.23 v_daily_order_stats

Dzienne zestawienie złożonych zamówień (ilość i wartość) oraz zamówień zakończonych.

```sql
CREATE OR ALTER VIEW v_daily_order_stats AS
SELECT
    CAST(o.order_date AS DATE) AS report_date,
    COUNT(DISTINCT o.order_id) AS total_orders_placed,
    ROUND(SUM(od.quantity * od.unit_price * (1 - o.order_discount)), 2) AS value_placed_today,
    (SELECT COUNT(DISTINCT os2.order_id)
     FROM order_status os2
     JOIN status_for_order s2 ON os2.status_id = s2.status_id
     WHERE s2.status_name = 'Completed'
       AND CAST(os2.end_date AS DATE) = CAST(o.order_date AS DATE)) AS orders_completed_today,
    (SELECT ROUND(SUM(od2.quantity * od2.unit_price * (1 - o2.order_discount)), 2)
     FROM order_details od2
     JOIN orders o2 ON od2.order_id = o2.order_id
     JOIN order_status os2 ON o2.order_id = os2.order_id
     JOIN status_for_order s2 ON os2.status_id = s2.status_id
     WHERE s2.status_name = 'Completed'
       AND CAST(os2.end_date AS DATE) = CAST(o.order_date AS DATE)) AS value_completed_today
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
GROUP BY CAST(o.order_date AS DATE);
```

3.24 v_order_financial_summary

Podsumowanie finansowe pojedynczego zamówienia (netto/brutto) wraz z aktualnym statusem.

```sql
CREATE OR ALTER VIEW v_order_financial_summary AS
SELECT
    o.order_id,
    s.status_name AS current_status,
    ROUND(SUM(od.quantity * od.unit_price * (1 - o.order_discount)), 2) AS total_net_value,
    ROUND(SUM((od.quantity * od.unit_price * (1 - o.order_discount)) * (1 + ISNULL(v.amount, 0))), 2) AS total_gross_value
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN order_status os ON o.order_id = os.order_id
JOIN status_for_order s ON os.status_id = s.status_id
LEFT JOIN products_vat pv ON od.product_id = pv.product_id
    AND o.order_date >= pv.start_date
    AND (pv.end_date IS NULL OR o.order_date <= pv.end_date)
LEFT JOIN vat v ON pv.vat_id = v.vat_id
WHERE os.order_status_id = (SELECT MAX(order_status_id) FROM order_status WHERE order_id = o.order_id)
GROUP BY o.order_id, s.status_name;
```

3.25 v_active_delayed_orders

Lista aktywnych zamówień, które przekroczyły planowany termin realizacji.

```sql
CREATE OR ALTER VIEW v_active_delayed_orders AS
SELECT
    o.order_id,
    o.order_date,
    os.planned_end_date AS current_deadline,
    s.status_name AS current_status,
    DATEDIFF(day, os.planned_end_date, GETDATE()) AS days_overdue
FROM orders o
JOIN order_status os ON o.order_id = os.order_id
JOIN status_for_order s ON os.status_id = s.status_id
WHERE os.order_status_id = (SELECT MAX(os2.order_status_id) FROM order_status os2 WHERE os2.order_id = o.order_id)
  AND s.status_name NOT IN ('Completed', 'Suspended', 'Cancelled', 'Return')
  AND os.end_date IS NULL
  AND os.planned_end_date < GETDATE();
```

3.26 v_orders_scheduled_today

Zamówienia, dla których bieżący etap rozpoczął się dzisiaj.

```sql
CREATE OR ALTER VIEW v_orders_scheduled_today AS
SELECT
    o.order_id,
    o.client_id,
    s.status_name,
    os.start_date AS status_started_at
FROM orders o
JOIN order_status os ON o.order_id = os.order_id
JOIN status_for_order s ON os.status_id = s.status_id
```

```sql
WHERE os.order_status_id = (SELECT MAX(os2.order_status_id) FROM order_status os2 WHERE os2.order_id = o.order_id)
  AND CAST(os.start_date AS DATE) = CAST(GETDATE() AS DATE);
```

3.27 v_total_orders_scheduled_per_day

Zestawienie obłożenia na przyszłe dni (liczba zamówień planowanych do zakończenia danego dnia).

```sql
CREATE OR ALTER VIEW v_total_orders_scheduled_per_day AS
SELECT
    CAST(os.planned_end_date AS DATE) AS planned_date,
    COUNT(DISTINCT os.order_id) AS total_orders_to_finish
FROM order_status os
WHERE os.order_status_id = (SELECT MAX(os2.order_status_id) FROM order_status os2 WHERE os2.order_id = os.order_id)
  AND os.end_date IS NULL -- Tylko trwajace etapy
GROUP BY CAST(os.planned_end_date AS DATE);
```

3.28 v_critical_orders_due_today

Zamówienia priorytetowe - termin realizacji mija dzisiaj.

```sql
CREATE OR ALTER VIEW v_critical_orders_due_today AS
SELECT
    o.order_id,
    COALESCE(cc.company_name, ci.first_name + ' ' + ci.last_name) AS client_name,
    os.planned_end_date,
    s.status_name
FROM orders o
JOIN order_status os ON o.order_id = os.order_id
JOIN status_for_order s ON os.status_id = s.status_id
LEFT JOIN clients_company cc ON o.client_id = cc.client_id
LEFT JOIN clients_individual ci ON o.client_id = ci.client_id
WHERE os.order_status_id = (SELECT MAX(os2.order_status_id) FROM order_status os2 WHERE os2.order_id = o.order_id)
  AND CAST(os.planned_end_date AS DATE) = CAST(GETDATE() AS DATE)
  AND s.status_name NOT IN ('Completed', 'Cancelled');
```

3.29 Widoki słownikowe UI

Zestaw prostych widoków ułatwiających pobieranie list rozwijanych w interfejsie aplikacji.

```sql
CREATE OR ALTER VIEW v_ui_product_categories AS
SELECT category_id, name AS category_name FROM categories_for_product;

CREATE OR ALTER VIEW v_ui_part_categories AS
SELECT category_id, name AS category_name FROM categories_for_part;

CREATE OR ALTER VIEW v_ui_employee_roles AS
SELECT role_id, possition_name, description FROM roles;

CREATE OR ALTER VIEW v_ui_production_statuses AS
SELECT status_id, status_name FROM status_for_production;

CREATE OR ALTER VIEW v_ui_order_statuses AS
SELECT status_id, status_name FROM status_for_order;
```

4 Logika Biznesowa: Funkcje, Wyzwalacze i Procedury

4.1 Funkcje (Functions)

4.1.1 fn_part_offers_by_deadline

Opis: Funkcja tabelaryczna, która filtruje oferty dostawców dla określonej części (`part_id`), wybierając tylko te, które mogą zostać zrealizowane (dostarczone) przed podaną datą (`deadline`). Jeśli data nie jest podana, zwraca wszystkie oferty dla części. Pomaga w szybkim wyborze dostawcy w sytuacjach krytycznych czasowo.

- **Parametry wejściowe:** `@part_id` (INT) - ID części, `@deadline` (DATE) - Oczekiwana data dostawy.

```sql
CREATE FUNCTION fn_part_offers_by_deadline (@part_id INT, @deadline DATE)
RETURNS TABLE
AS
RETURN (
    SELECT
        p.part_name,
        s.company_name,
        ROUND(so.price, 2) AS price,
        so.delivery_days,
        DATEADD(day, so.delivery_days, GETDATE()) AS potential_delivery_date
    FROM supplier_offers so
    JOIN parts p ON so.part_id = p.part_id
    JOIN suppliers s ON so.supplier_id = s.supplier_id
    WHERE so.part_id = @part_id
      AND (@deadline IS NULL OR DATEADD(day, so.delivery_days, GETDATE()) <= @deadline)
);
```

4.1.2 fn_get_supplier_offers_by_name_or_id

Opis: Wyszukuje pełną listę ofert konkretnego dostawcy. Umożliwia wyszukiwanie zarówno po identyfikatorze (`supplier_id`), jak i po nazwie firmy (wyszukiwanie rozmyte operatorem `LIKE`). Ułatwia przegląd cennika konkretnego kontrahenta.

- **Parametry wejściowe:** `@supplier_info` (NVARCHAR) - Nazwa firmy lub ID dostawcy.

```sql
CREATE FUNCTION fn_get_supplier_offers_by_name_or_id (
    @supplier_info NVARCHAR(255)
)
RETURNS TABLE
AS
RETURN (
    SELECT
        s.supplier_id,
        s.company_name,
        p.part_name,
        ROUND(so.price, 2) AS price,
        so.delivery_days,
        so.pack_size
    FROM supplier_offers so
    JOIN suppliers s ON so.supplier_id = s.supplier_id
    JOIN parts p ON so.part_id = p.part_id
    WHERE
```

```
        CAST(s.supplier_id AS NVARCHAR) = @supplier_info
        OR s.company_name LIKE '%' + @supplier_info + '%'
);
```

4.1.3 fn_get_late_orders_up_to_date

Opis: Identyfikuje zamówienia części od dostawców, które są opóźnione względem planu. Zwraca zamówienia, których planowana data dostawy była wcześniejsza lub równa zadanej dacie (`deadline_date`) a które nie zostały jeszcze oznaczone jako dostarczone, lub zostały dostarczone po terminie.

- **Parametry wejściowe:** `@deadline_date` (DATE) - Data graniczna.

```
CREATE FUNCTION fn_get_late_orders_up_to_date (@deadline_date DATE)
RETURNS TABLE
AS
RETURN (
    SELECT
        op.order_part_id,
        p.part_name,
        s.company_name AS supplier_name,
        op.order_date,
        DATEADD(day, so.delivery_days, op.order_date) AS planned_delivery_date,
        op.order_delivered AS actual_delivery_date,
        DATEDIFF(day, DATEADD(day, so.delivery_days, op.order_date), ISNULL(op.
    order_delivered, GETDATE())) AS delay_days,
        CASE
            WHEN op.order_delivered IS NULL THEN 'Zaległe (nie dostarczone)'
            ELSE 'Dostarczone po terminie'
        END AS status_opoznienia
    FROM order_parts op
    JOIN supplier_offers so ON op.offer_id = so.offer_id
    JOIN suppliers s ON so.supplier_id = s.supplier_id
    JOIN parts p ON so.part_id = p.part_id
    WHERE
        DATEADD(day, so.delivery_days, op.order_date) <= @deadline_date
        AND (
            op.order_delivered > DATEADD(day, so.delivery_days, op.order_date)
            OR (op.order_delivered IS NULL AND DATEADD(day, so.delivery_days, op.
    order_date) < GETDATE())
        )
);
```

4.1.4 fn_get_employee_future_time_offs

Opis: Wyświetla listę przyszłych, zaplanowanych nieobecności (urlopów, zwolnień) dla danego pracownika. Służy do weryfikacji dostępności zasobów ludzkich przed przypisaniem ich do nowych zleceń produkcyjnych.

- **Parametry wejściowe:** `@search_val` (NVARCHAR) - ID pracownika lub jego adres e-mail.

```
CREATE FUNCTION fn_get_employee_future_time_offs (
    @search_val NVARCHAR(150)
)
RETURNS TABLE
AS
RETURN (
```

```sql
    SELECT
        e.first_name,
        e.last_name,
        eto.start_date,
        eto.end_date,
        DATEDIFF(day, eto.start_date, eto.end_date) + 1 AS duration_days,
        eto.reason
    FROM employee_time_offs eto
    JOIN employees e ON eto.employee_id = e.employee_id
    WHERE (CAST(e.employee_id AS NVARCHAR) = @search_val OR e.email = @search_val)
      AND eto.end_date >= CAST(GETDATE() AS DATE)
);
```

4.1.5 fn_get_revenue_report

Opis: Uniwersalna funkcja raportowa do analizy przychodów. Pozwala wygenerować raport za konkretny rok i (opcjonalnie) miesiąc. Umożliwia agregację danych na poziomie kategorii produktów (by_categories=1) lub poszczególnych produktów, a także rozbicie czasowe na tygodnie (by_weeks=1).

- **Parametry wejściowe:** `@year` (INT), `@month` (INT, opcjonalny), `@by_categories` (BIT), `@by_weeks` (BIT).

```sql
CREATE FUNCTION fn_get_revenue_report (
    @year INT,
    @month INT,          -- NULL = raport roczny
    @by_categories BIT = 1, -- 1 = grupuj kategoriami ; 0 = grupuj produktami
    @by_weeks BIT = 1       -- 1 = rozbij na tygodnie ; 0 = nie rozbijaj
)
RETURNS TABLE
AS
RETURN (
    SELECT
        CASE WHEN @by_categories = 1 THEN c.category_id ELSE p.product_id END AS group_id,
        CASE WHEN @by_categories = 1 THEN c.name ELSE p.product_name END AS group_name,
        YEAR(o.order_date) AS report_year,
        CASE WHEN @month IS NOT NULL THEN MONTH(o.order_date) ELSE NULL END AS report_month,
        CASE WHEN @by_weeks = 1 THEN DATEPART(week, o.order_date) ELSE NULL END AS report_week,
        SUM(od.quantity) AS total_units_sold,
        ROUND(SUM(od.quantity * od.unit_price * (1 - o.order_discount)), 2) AS total_revenue,
        COUNT(DISTINCT o.order_id) AS number_of_orders
    FROM products p
    JOIN order_details od ON p.product_id = od.product_id
    JOIN orders o ON od.order_id = o.order_id
    JOIN categories_for_product c ON p.category_id = c.category_id
    WHERE YEAR(o.order_date) = @year
      AND (@month IS NULL OR MONTH(o.order_date) = @month)
    GROUP BY
        CASE WHEN @by_categories = 1 THEN c.category_id ELSE p.product_id END,
        CASE WHEN @by_categories = 1 THEN c.name ELSE p.product_name END,
        YEAR(o.order_date),
        CASE WHEN @month IS NOT NULL THEN MONTH(o.order_date) ELSE NULL END,
        CASE WHEN @by_weeks = 1 THEN DATEPART(week, o.order_date) ELSE NULL END
);
```

4.1.6 fn_production_cost_profit_report

Opis: Kompleksowa funkcja analityczna (BI) obliczająca rentowność produkcji. Zestawia przychody ze sprzedaży z kosztami produkcji (koszt materiałów BOM + koszt roboczogodzin). Pozwala na analizę w ujęciu rocznym, kwartalnym, miesięcznym lub tygodniowym. Kluczowa dla oceny kondycji finansowej działu produkcji.

- **Parametry wejściowe:** `@group_by_category` (BIT), `@period_type` .

```sql
CREATE OR ALTER FUNCTION fn_production_cost_profit_report (
    @group_by_category BIT = 1, -- 1 = grupuj kategoriami ; 0 = grupuj produktami
    @period_type INT = 2 -- 1 = kwartał ; 2 = miesiąc ; 3 = tydzień
)
RETURNS TABLE
AS
RETURN
(
    WITH PartPrices AS (
        SELECT part_id, AVG(price) AS avg_part_price
        FROM supplier_offers
        GROUP BY part_id
    ),
    ProductCosts AS (
        SELECT
            p.product_id,
            p.category_id,
            p.product_name,
            p.needed_hours,
            ISNULL(SUM(pr.quantity_needed * pp.avg_part_price), 0) AS unit_parts_cost
        FROM products p
        LEFT JOIN product_recipes pr ON p.product_id = pr.product_id
        LEFT JOIN PartPrices pp ON pr.part_id = pp.part_id
        GROUP BY p.product_id, p.category_id, p.product_name, p.needed_hours
    ),
    RawData AS (
        SELECT
            product_id,
            YEAR(o.order_date) AS r_year,
            CASE
                WHEN @period_type = 1 THEN CAST(DATEPART(QUARTER, o.order_date) AS VARCHAR)
                WHEN @period_type = 2 THEN CAST(MONTH(o.order_date) AS VARCHAR)
                WHEN @period_type = 3 THEN CAST(DATEPART(WEEK, o.order_date) AS VARCHAR)
                ELSE '0'
            END AS r_period,
            od.quantity AS sold_qty,
            0 AS prod_qty,
            (od.quantity * od.unit_price * (1 - ISNULL(o.order_discount, 0))) AS revenue,
            0 AS calculated_prod_cost,
            0 AS calculated_parts_cost
        FROM order_details od
        JOIN orders o ON od.order_id = o.order_id

        UNION ALL

        SELECT
            p.product_id,
            YEAR(ps.start_date) AS r_year,
            CASE
                WHEN @period_type = 1 THEN CAST(DATEPART(QUARTER, ps.start_date) AS VARCHAR)
```

```sql
                WHEN @period_type = 2 THEN CAST(MONTH(ps.start_date) AS VARCHAR)
                WHEN @period_type = 3 THEN CAST(DATEPART(WEEK, ps.start_date) AS VARCHAR)
                ELSE '0'
            END AS r_period,
            0 AS sold_qty,
            p.quantity AS prod_qty,
            0 AS revenue,
            (p.quantity * (pc.unit_parts_cost + (pc.needed_hours * e.base_salary))) AS calculated_prod_cost,
            (p.quantity * pc.unit_parts_cost) AS calculated_parts_cost
        FROM productions p
        JOIN production_status ps ON p.production_id = ps.production_id
        JOIN employees e ON p.supervisor_id = e.employee_id
        JOIN ProductCosts pc ON p.product_id = pc.product_id
    ),
    AggregatedData AS (
        SELECT
            product_id,
            r_year,
            r_period,
            SUM(sold_qty) AS total_sold,
            SUM(prod_qty) AS total_produced,
            SUM(revenue) AS total_revenue,
            SUM(calculated_prod_cost) AS total_cost,
            SUM(calculated_parts_cost) AS total_parts_cost
        FROM RawData
        GROUP BY product_id, r_year, r_period
    )
    SELECT
        CASE WHEN @group_by_category = 1 THEN CAST(pc.category_id AS VARCHAR) ELSE CAST(pc.product_id AS VARCHAR) END AS [ID],
        CASE WHEN @group_by_category = 1 THEN c.name ELSE pc.product_name END AS [Name],
        ad.r_year AS [Year],
        CASE
            WHEN @period_type = 1 THEN 'Q' + ad.r_period
            WHEN @period_type = 2 THEN DATENAME(month, DATEFROMPARTS(ad.r_year, CAST(ad.r_period AS INT), 1))
            WHEN @period_type = 3 THEN 'Week ' + ad.r_period
            ELSE 'Annual'
        END AS [Period_Label],

        SUM(ad.total_sold) AS [Total_Sold_Units],
        SUM(ad.total_produced) AS [Total_Produced_Units],
        CAST(ROUND(SUM(ad.total_produced * pc.needed_hours), 2) AS DECIMAL(18,2)) AS [Total_Produced_Working_Hours],
        CAST(ROUND(SUM(ad.total_parts_cost), 2) AS DECIMAL(18,2)) AS [Total_Parts_Cost],
        CAST(ROUND(SUM(ad.total_revenue), 2) AS DECIMAL(18,2)) AS [Total_Sales],
        CAST(ROUND(SUM(ad.total_cost), 2) AS DECIMAL(18,2)) AS [Total_Production_Costs],
        CAST(ROUND(SUM(ad.total_revenue) - SUM(ad.total_cost), 2) AS DECIMAL(18,2)) AS [Net_Profit]
    FROM AggregatedData ad
    JOIN ProductCosts pc ON ad.product_id = pc.product_id
    JOIN categories_for_product c ON pc.category_id = c.category_id
    GROUP BY
        CASE WHEN @group_by_category = 1 THEN CAST(pc.category_id AS VARCHAR) ELSE CAST(pc.product_id AS VARCHAR) END,
        CASE WHEN @group_by_category = 1 THEN c.name ELSE pc.product_name END,
        ad.r_year,
        ad.r_period
);
```

4.1.7 fn_production_costs

Opis: Uproszczona wersja raportu kosztów produkcji, zwracająca tylko pozycje, dla których zarejestrowano produkcję.

- **Parametry wejściowe:** Analogiczne do `fn_production_cost_profit_report`.

```sql
create or alter function fn_production_costs (
    @group_by_category BIT = 1, -- 1 = Wyniki per Kategoria ; 0 = Wyniki per Produkt (jednostkowo)
    @period_type INT = 2        -- 0 = Rok ; 1 = Kwartal ; 2 = Miesiac ; 3 = Tydzien

)
returns table as
return
(
    select ID,Name,Year,Period_Label,Total_Produced_Units,Total_Produced_Working_Hours,Total_Parts_Cost,Total_Production_Costs from fn_production_cost_profit_report(
        @group_by_category,
        @period_type,
        @work_hour_price
    ) where Total_Produced_Units>0
);
```

4.1.8 fn_production_in_progress

Opis: Zwraca listę zleceń produkcyjnych, które znajdują się w toku (rozpoczęte, ale niezakończone) w zadanym przedziale czasowym. Służy do monitorowania bieżącego obłożenia hali produkcyjnej.

- **Parametry wejściowe:** `@start_date` (DATETIME) - lewy brzeg wyszukiwania, @end_date (DATETIME) - prawy brzeg wyszukiwania.

```sql
create or alter function fn_production_in_progress(
  @start_date DATETIME = '0000-00-00',
  @end_date DATETIME
)
returns table as
return (
  select production_id,product_name,quantity,start_date,planned_end_date,supervisor_id,delay_days,status_delay from v_production_delay_details where actual_end_date is null and work_place_id is not null
  and start_date>=@start_date and planned_end_date<=@end_date
);
```

4.1.9 fn_estimate_order_delivery

Opis: Szacuje datę dostawy na podstawie dostępności produktów i czasu produkcji.

- **Parametry wejściowe:** `@order_id` (INT), `@shipping_buffer` (INT, domyślnie 2 dni).
- **Działanie:** Dla każdego produktu w zamówieniu sprawdza jego dostępność. Jeśli brakuje towaru, oblicza czas potrzebny na wyprodukowanie brakującej ilości i sprowadzenie części. Jako datę końcową przyjmuje najpóźniejszy termin gotowości wszystkich produktów powiększony o bufor wysyłkowy.

```sql
CREATE OR ALTER FUNCTION fn_estimate_order_delivery (
    @order_id INT,
    @shipping_buffer INT = 2
)
RETURNS DATE
AS
BEGIN
    DECLARE @final_date DATE;

    SELECT @final_date = MAX(ItemCompletionDate)
    FROM (
        SELECT
            CASE
                WHEN ip.available_units >= od.quantity THEN GETDATE()
                ELSE
                    DATEADD(DAY,
                        ISNULL((
                            SELECT MAX(so.delivery_days)
                            FROM product_recipes pr
                            JOIN supplier_offers so ON pr.part_id = so.part_id
                            WHERE pr.product_id = od.product_id
                              AND (
                                   SELECT available_units FROM inventory_parts WHERE part_id = pr.part_id
                                  ) < (pr.quantity_needed * od.quantity)
                        ), 0) +
                        CEILING((CAST(od.quantity AS FLOAT) * ISNULL(p_info.needed_hours, 1)) / 24.0),
                        GETDATE()
                    )
            END AS ItemCompletionDate
        FROM order_details od
        LEFT JOIN inventory_products ip ON od.product_id = ip.product_id
        LEFT JOIN products p_info ON od.product_id = p_info.product_id
        WHERE od.order_id = @order_id
    ) AS ItemEstimates;

    RETURN DATEADD(DAY, @shipping_buffer, @final_date);
END;
```

4.1.10 fn_get_client_purchase_history

Opis: Zwraca historię zakupów klienta w zadanym przedziale czasowym.

- **Parametry wejściowe:** `@client_info` (NVARCHAR) - ID lub email, `@date_from` (DATE), `@date_to` (DATE).
- **Działanie:** Pobiera szczegółową listę wszystkich pozycji zakupionych przez klienta, uwzględniając rabaty.

```sql
CREATE OR ALTER FUNCTION fn_get_client_purchase_history (
    @client_info NVARCHAR(255),
    @date_from DATE
)
RETURNS TABLE
AS
RETURN (
    SELECT
        o.client_id,
```

```sql
        c.email AS client_email,
        o.order_id,
        o.order_date,
        p.product_name,
        od.quantity,
        CAST(ROUND(od.unit_price, 2) AS DECIMAL(18,2)) AS unit_price,
        o.order_discount,
        CAST(ROUND((od.quantity * od.unit_price * (1 - ISNULL(o.order_discount, 0))), 2)
    AS DECIMAL(18,2)) AS final_line_value
    FROM orders o
    JOIN clients c ON o.client_id = c.client_id
    JOIN order_details od ON o.order_id = od.order_id
    JOIN products p ON od.product_id = p.product_id
    WHERE
        (
            CAST(o.client_id AS NVARCHAR) = @client_info
            OR
            c.email = @client_info
        )
        AND CAST(o.order_date AS DATE) >= @date_from
        AND o.order_date <= GETDATE()
);
```

4.1.11 fn_get_production_team

Opis: Zwraca listę pracowników przypisanych do konkretnej produkcji wraz z ich rolami.

- **Parametry wejściowe:** `@target_production_id` (INT).

```sql
CREATE OR ALTER FUNCTION fn_get_production_team (
    @target_production_id INT
)
RETURNS TABLE
AS
RETURN (
    SELECT
        p.production_id,
        pr.product_name,
        e.employee_id,
        e.first_name + ' ' + e.last_name AS employee_full_name,
        r.possition_name AS employee_role,
        pta.hours_logged,
        -- Dane kierownika produkcji
        p.supervisor_id AS production_supervisor_id,
        sup.first_name + ' ' + sup.last_name AS production_supervisor_name
    FROM productions p
    JOIN products pr ON p.product_id = pr.product_id
    JOIN production_team_assignments pta ON p.production_id = pta.production_id
    JOIN employees e ON pta.employee_id = e.employee_id
    JOIN roles r ON e.role_id = r.role_id
    JOIN employees sup ON p.supervisor_id = sup.employee_id
    WHERE p.production_id = @target_production_id
);
```

4.1.12 fn_get_employee_production_assignments

Opis: Pobiera wszystkie aktywne przypisania pracownika do procesów produkcyjnych.

- **Parametry wejściowe:** `@employee_info` (NVARCHAR) - employee_id / email.

```sql
CREATE OR ALTER FUNCTION fn_get_employee_production_assignments (
    @employee_info NVARCHAR(255)
)
RETURNS TABLE
AS
RETURN (
    SELECT
        e.employee_id,
        e.first_name + ' ' + e.last_name AS employee_name,
        e.email,
        p.production_id,
        pr.product_name,
        p.quantity AS quantity_to_make,
        s.status_name AS production_current_status,
        pta.hours_logged,
        p.supervisor_id,
        CONCAT(e2.first_name,' ',e2.last_name) as production_supervisor_name,
        e2.email as production_supervisor_email
    FROM employees e
    JOIN production_team_assignments pta ON e.employee_id = pta.employee_id
    JOIN productions p ON pta.production_id = p.production_id
    JOIN products pr ON p.product_id = pr.product_id
    JOIN production_status ps ON p.production_id = ps.production_id
    JOIN status_for_production s ON ps.status_id = s.status_id
    JOIN employees as e2 on e2.employee_id=p.supervisor_id
    WHERE
        (CAST(e.employee_id AS NVARCHAR) = @employee_info OR e.email = @employee_info)
        AND ps.end_date IS NULL
);
```

4.1.13 fn_get_product_recipe_details

Opis: Funkcja pomocnicza zwracająca szczegóły receptury (BOM) dla zadanego produktu (po ID lub nazwie).

- **Parametry wejściowe:** `@product_id_or_name` (NVARCHAR) - product_id / product_name.

```sql
CREATE OR ALTER FUNCTION fn_get_product_recipe_details (
    @product_id_or_name nvarchar(100)
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        p.part_id AS [ID],
        p.part_name AS [Nazwa_Czesci],
        pr.quantity_needed AS [Wymagana_Ilosc],
        cp.name AS [Kategoria_Czesci],
        prod.product_name AS [Dla_Produktu]
    FROM products prod
    JOIN product_recipes pr ON prod.product_id = pr.product_id
    JOIN parts p ON pr.part_id = p.part_id
    LEFT JOIN categories_for_part cp ON p.category_id = cp.category_id
    WHERE
        prod.product_name = @product_id_or_name
        OR CAST(prod.product_id AS nvarchar(100)) = @product_id_or_name
);
```

4.2 Wyzwalacze (Triggers)

4.2.1 trg_update_stock_on_delivery

Opis: Trigger uruchamiany po aktualizacji tabeli `order_parts`. Monitoruje zmianę kolumny `order_delivered`. W momencie, gdy zamówienie zostaje oznaczone jako dostarczone (zmiana z NULL na datę), trigger automatycznie zwiększa stan magazynowy odpowiednich części w tabeli `inventory_parts`. Ilość dodawana do magazynu to iloczyn liczby zamówionych paczek (`pack_quantity`) i wielkości paczki (`pack_size`) z oferty. Zapewnia to automatyczną synchronizację stanów magazynowych z procesem zakupowym.

```sql
CREATE TRIGGER trg_update_stock_on_delivery
ON order_parts
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF UPDATE(order_delivered)
    BEGIN
        UPDATE ip
        SET ip.available_units = ip.available_units + (i.pack_quantity * so.pack_size)
        FROM inventory_parts ip
        JOIN supplier_offers so ON ip.part_id = so.part_id
        JOIN inserted i ON i.offer_id = so.offer_id
        JOIN deleted d ON i.order_part_id = d.order_part_id
        WHERE i.order_delivered IS NOT NULL AND d.order_delivered IS NULL;
    END
END;
```

4.2.2 trg_auto_plan_production

Opis: Trigger uruchamiany po aktualizacji stanu magazynowego produktów (`inventory_products`). Jego celem jest automatyzacja planowania produkcji. Gdy stan dostępny produktu spadnie poniżej zdefiniowanego minimum i nie ma aktualnie trwającej ani zaplanowanej produkcji tego produktu, trigger: 1. Oblicza brakującą ilość towaru potrzebną do zapełnienia magazynu w 80% jego pojemności maksymalnej. 2. Szacuje czas potrzebny na wyprodukowanie tej partii (na podstawie needed_hours z definicji produktu). 3. Automatycznie tworzy nowe zlecenie produkcyjne w tabeli `productions` (ze statusem 'Planned'). Dzięki temu system samodzielnie reaguje na braki magazynowe, generując propozycje planów produkcyjnych.

```sql
CREATE OR ALTER TRIGGER trg_auto_plan_production
ON inventory_products
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @prod_id INT, @current_stock INT, @min_units INT, @max_units INT;
    DECLARE prod_cursor CURSOR FOR
    SELECT product_id, available_units, min_units, max_units FROM inserted;

    OPEN prod_cursor;
    FETCH NEXT FROM prod_cursor INTO @prod_id, @current_stock, @min_units, @max_units;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        --stan ponizej min oraz brak aktywnej produkcji
        IF @current_stock <= @min_units
```

```
            AND NOT EXISTS (
                SELECT 1 FROM productions p
                JOIN production_status ps ON p.production_id = ps.production_id
                WHERE p.product_id = @prod_id AND ps.status_id IN (1, 2) AND ps.end_date IS NULL
            )
        BEGIN
            -- ile wyprodukowac by zapchac 80% magazynu
            DECLARE @qty_to_produce INT = (CAST(@max_units AS FLOAT) * 0.8) - @current_stock;

            IF @qty_to_produce > 0
            BEGIN
                --Pobranie godzin z tabeli products i obliczenie dni
                DECLARE @hours_per_unit FLOAT;
                SELECT @hours_per_unit = needed_hours FROM products WHERE product_id = @prod_id;

                -- default planned_date for planned status (godziny * ilosc) / 24
                DECLARE @total_hours FLOAT = ISNULL(@hours_per_unit, 1) * @qty_to_produce;
                DECLARE @days_needed INT = CEILING(@total_hours / 24.0);

                DECLARE @new_production_id INT;
                INSERT INTO productions (product_id, quantity, supervisor_id, work_place_id)
                VALUES (@prod_id, @qty_to_produce, 1, NULL); -- emplyee id 1 as default

                SET @new_production_id = SCOPE_IDENTITY();
                INSERT INTO production_status (production_id, status_id, start_date, planned_end_date)
                VALUES (
                    @new_production_id,
                    1,                  -- Status: Planned
                    GETDATE(),
                    DATEADD(day, @days_needed, GETDATE())
                );
            END
        END
        FETCH NEXT FROM prod_cursor INTO @prod_id, @current_stock, @min_units, @max_units;
    END
    CLOSE prod_cursor; DEALLOCATE prod_cursor;
END;
```

4.2.3 trg_auto_order_parts

Opis: Trigger odpowiedzialny za ciągłość łańcucha dostaw. Działa na tabeli `inventory_parts`. Gdy stan surowca/części spadnie poniżej minimum i nie ma aktywnego (niedostarczonego) zamówienia na ten surowiec, trigger: 1. Oblicza ilość potrzebną do uzupełnienia stanu do poziomu 80% maksimum. 2. Automatycznie wyszukuje **najkorzystniejszą ofertę** w tabeli `supplier_offers` (sortując rosnąco po cenie, a następnie po czasie dostawy). 3. Tworzy nowe zamówienie w `order_parts`, przeliczając wymaganą ilość sztuk na pełne opakowania (paczki). Mechanizm ten zapobiega przestojom w produkcji spowodowanym brakiem komponentów.

```
CREATE TRIGGER trg_auto_order_parts
ON inventory_parts
AFTER UPDATE
AS
BEGIN
```

```sql
    SET NOCOUNT ON;

    DECLARE @part_id INT, @current_stock INT, @min_units INT, @max_units INT;
    DECLARE part_cursor CURSOR FOR
    SELECT i.part_id, i.available_units, i.min_units, i.max_units FROM inserted i;

    OPEN part_cursor;
    FETCH NEXT FROM part_cursor INTO @part_id, @current_stock, @min_units, @max_units;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        --Stan <= Min ORAZ brak niedostarczonych (ale aktywnych) zamowien na te czesc
        IF @current_stock <= @min_units
           AND NOT EXISTS (
               SELECT 1 FROM order_parts op
               JOIN supplier_offers so ON op.offer_id = so.offer_id
               WHERE so.part_id = @part_id AND op.order_delivered IS NULL
           )
        BEGIN
            -- ile zamowic aby wypchac 80% magazynu
            DECLARE @target_qty INT = (CAST(@max_units AS FLOAT) * 0.8) - @current_stock;

            IF @target_qty > 0
            BEGIN
                -- wyszukanie najlepszej oferty na te czesc
                DECLARE @best_offer_id INT, @pack_size INT;
                SELECT TOP 1 @best_offer_id = offer_id, @pack_size = pack_size
                FROM supplier_offers WHERE part_id = @part_id ORDER BY price ASC, delivery_days ASC;

                IF @best_offer_id IS NOT NULL
                BEGIN
                    DECLARE @packs_to_order INT = CEILING(CAST(@target_qty AS FLOAT) / @pack_size);
                    INSERT INTO order_parts (offer_id, order_date, pack_quantity, order_delivered)
                    VALUES (@best_offer_id, GETDATE(), @packs_to_order, NULL);
                END
            END
        END
        FETCH NEXT FROM part_cursor INTO @part_id, @current_stock, @min_units, @max_units;
    END
    CLOSE part_cursor; DEALLOCATE part_cursor;
END;
```

4.3 Procedury Składowane (Stored Procedures)

4.3.1 sp_upsert_supplier

Opis: Procedura służąca do zarządzania bazą dostawców. Realizuje logikę "Update or Insert". Sprawdza, czy dostawca o podanym adresie email już istnieje. Jeśli tak – aktualizuje jego nazwę. Jeśli nie – tworzy nowy wpis w tabeli `suppliers`. Zapobiega to duplikowaniu rekordów dostawców. **Parametry:**

- `@company_name` (NVARCHAR): Nazwa firmy.
- `@email` (NVARCHAR): Email (unikalny klucz).

```sql
CREATE PROCEDURE sp_upsert_supplier
    @company_name NVARCHAR(255),
    @email NVARCHAR(150)
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM suppliers WHERE email = @email)
    BEGIN
        UPDATE suppliers
        SET company_name = @company_name
        WHERE email = @email;
    END
    ELSE
    BEGIN
        INSERT INTO suppliers (company_name, email)
        VALUES (@company_name, @email);
    END
END;
```

4.3.2 sp_upsert_supplier_offer

Opis: Zarządza ofertami handlowymi przypisanymi do dostawców i części. Pozwala zdefiniować lub zaktualizować cenę, czas dostawy oraz wielkość opakowania dla konkretnej pary Dostawca-Część. Jeśli oferta już istnieje, jest aktualizowana; w przeciwnym razie tworzona jest nowa. Wymaga podania ceny i czasu dostawy przy tworzeniu nowej oferty.

```sql
CREATE PROCEDURE sp_upsert_supplier_offer
    @supplier_id INT,
    @part_id INT,

    -- opcjonalne
    @price MONEY = NULL,
    @delivery_days INT = NULL,
    @pack_size INT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- czy oferta juz istnieje?
    IF EXISTS (SELECT 1 FROM supplier_offers WHERE supplier_id = @supplier_id AND part_id = @part_id)
    BEGIN
        -- scenario: update
        UPDATE supplier_offers
        SET price = ISNULL(@price, price),
            delivery_days = ISNULL(@delivery_days, delivery_days),
            pack_size = ISNULL(@pack_size, pack_size)
        WHERE supplier_id = @supplier_id AND part_id = @part_id;
    END
    ELSE
    BEGIN
        -- scenario: insert
        IF @price IS NULL OR @delivery_days IS NULL
        BEGIN
            RAISERROR('Błąd: Tworząc nową ofertę musisz podać cenę i czas dostawy.', 16, 1);
            RETURN;
        END
        INSERT INTO supplier_offers (supplier_id, part_id, price, delivery_days, pack_size)
```

```
        VALUES (@supplier_id, @part_id, @price, @delivery_days, ISNULL(@pack_size, 1));
    END
END;
```

4.3.3 sp_add_new_part_type

Opis: Procedura ułatwiająca wprowadzanie nowych typów części do systemu. Przyjmuje nazwę części, nazwę kategorii oraz parametry magazynowe. Jeśli podana kategoria nie istnieje, procedura automatycznie ją tworzy w tabeli `categories_for_part`. Następnie dodaje definicję części do `parts` oraz inicjalizuje rekord w `inventory_parts` (z zerowym stanem, ale ustawionymi limitami min/max).

```
CREATE PROCEDURE sp_add_new_part_type
    @part_name NVARCHAR(255),
    @category_name NVARCHAR(100),
    @unit_space_needed INT = 1,
    @min_units INT,
    @max_units INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        DECLARE @cat_id INT;

        -- jezeli kategoria nie istnieje to trzeba ja dodac
        SELECT @cat_id = category_id FROM categories_for_part WHERE name = @category_name
    ;

        IF @cat_id IS NULL
        BEGIN
            INSERT INTO categories_for_part (name) VALUES (@category_name);
            SET @cat_id = SCOPE_IDENTITY();
        END

        -- dodaj czesc do katalogu czesci
        INSERT INTO parts (part_name, category_id, unit_space_needed)
        VALUES (@part_name, @cat_id, @unit_space_needed);

        DECLARE @new_part_id INT = SCOPE_IDENTITY();

        -- dodaj wpis do inwentarza czesci, domyslnie 0 sztuk dostepnych
        INSERT INTO inventory_parts (part_id, available_units, min_units, max_units)
        VALUES (@new_part_id, 0, @min_units, @max_units);

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        -- blad transakcji rzucamy dalej
        THROW;
    END CATCH
END;
```

4.3.4 sp_place_part_order

Opis: Umożliwia manualne złożenie zamówienia na części u dostawcy. Kluczową funkcją tej procedury jest walidacja pojemności magazynu. Sprawdza, czy suma obecnego stanu, zamówień

będących już w drodze oraz nowego zamówienia nie przekroczy zdefiniowanego maksimum (max_units) dla danej części. Jeśli limit zostanie przekroczony, zamówienie jest blokowane z odpowiednim komunikatem błędu.

```sql
CREATE PROCEDURE sp_place_part_order
    @offer_id INT,
    @num_packs INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @part_id INT, @pack_size INT, @max_units INT, @current_stock INT, @pending_stock INT;
    SELECT
        @part_id = so.part_id,
        @pack_size = so.pack_size,
        @max_units = ip.max_units,
        @current_stock = ip.available_units
    FROM supplier_offers so
    JOIN inventory_parts ip ON so.part_id = ip.part_id
    WHERE so.offer_id = @offer_id;

    -- ile jest w drodze?
    SELECT @pending_stock = ISNULL(SUM(op.pack_quantity * so2.pack_size), 0)
    FROM order_parts op
    JOIN supplier_offers so2 ON op.offer_id = so2.offer_id
    WHERE so2.part_id = @part_id AND op.order_delivered IS NULL;

    -- stan obecny + w drodze + nowe zamowienie
    DECLARE @total_after_delivery INT = @current_stock + @pending_stock + (@num_packs * @pack_size);

    -- walidacja czy nie przekroczymy max_units
    IF @total_after_delivery > @max_units
    BEGIN
    -- formatmessage - komunikat, formatowanie tak jak w C
        DECLARE @msg NVARCHAR(255) = FORMATMESSAGE('Błąd: Brak miejsca. max: %d, obecnie+ w drodze: %d, próba zamówienia: %d.',
                                                   @max_units, (@current_stock + @pending_stock), (@num_packs * @pack_size));
        RAISERROR(@msg, 16, 1);
        RETURN;
    END

    -- finito - zloz zamowienie
    INSERT INTO order_parts (offer_id, order_date, pack_quantity, order_delivered)
    VALUES (@offer_id, GETDATE(), @num_packs, NULL);
END;
```

4.3.5 sp_adjust_part_inventory

Opis: Procedura serwisowa do ręcznej korekty stanów magazynowych (np. w przypadku inwentaryzacji, kradzieży lub zniszczenia towaru). Przyjmuje zmianę ilości (dodatnią lub ujemną). Zawiera zabezpieczenie przed ustawieniem ujemnego stanu końcowego – w takim przypadku operacja jest odrzucana.

```sql
CREATE OR ALTER PROCEDURE sp_adjust_part_inventory
    @part_id INT,
    @quantity_change INT -- moze byc ujemne!!
AS
BEGIN
```

```
    SET NOCOUNT ON;

    -- nie moze zejsc ponizej 0!!
    IF EXISTS (
        SELECT 1 FROM inventory_parts
        WHERE part_id = @part_id AND (available_units + @quantity_change) < 0
    )
    BEGIN
        RAISERROR('Błąd: Operacja spowodowałaby ujemny stan magazynowy.', 16, 1);
        RETURN;
    END

    UPDATE inventory_parts
    SET available_units = available_units + @quantity_change
    WHERE part_id = @part_id;
END;
```

4.3.6 sp_add_new_client

Opis: Procedura rejestracji nowego klienta, obsługująca logikę dziedziczenia tabel w bazie danych. Na podstawie flagi `@is_company` (1 - firma, 0 - osoba prywatna) procedura wstawia rekord do tabeli głównej `clients`, a następnie uzupełnia odpowiednią tabelę szczegółową (`clients_company` z NIP-em lub `clients_individual` z imieniem i nazwiskiem). Zapewnia spójność danych klienta w jednym kroku.

```
CREATE PROCEDURE sp_add_new_client
    @email NVARCHAR(150),
    @phone NVARCHAR(12),
    @city NVARCHAR(100),
    @address NVARCHAR(150),
    @city_code NVARCHAR(10),
    @is_company BIT, -- 1 = firma, 0 = osoba prywatna
    -- pola opcjonalne zalezne od typu
    @company_name NVARCHAR(255) = NULL,
    @nip NVARCHAR(10) = NULL,
    @first_name NVARCHAR(100) = NULL,
    @last_name NVARCHAR(100) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        -- walidacja danych zaleznych od wybranego typu klienta
        IF @is_company = 1 AND (@company_name IS NULL OR @nip IS NULL)
        BEGIN
            RAISERROR('Dla klienta firmowego wymagane są nazwa firmy i NIP.', 16, 1);
            RETURN;
        END

        IF @is_company = 0 AND (@first_name IS NULL OR @last_name IS NULL)
        BEGIN
            RAISERROR('Dla klienta indywidualnego wymagane są imię i nazwisko.', 16, 1);
            RETURN;
        END

        -- tabela wspolna
        INSERT INTO clients (email, phone, city, address, city_code)
        VALUES (@email, @phone, @city, @address, @city_code);

        DECLARE @new_client_id INT = SCOPE_IDENTITY();

```

```sql
        -- wstawienie do klientow indywidualnych lub firmowych
        IF @is_company = 1
        BEGIN
            INSERT INTO clients_company (client_id, company_name, nip)
            VALUES (@new_client_id, @company_name, @nip);
        END
        ELSE
        BEGIN
            INSERT INTO clients_individual (client_id, first_name, last_name)
            VALUES (@new_client_id, @first_name, @last_name);
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.7 sp_update_client_data

Opis: Aktualizuje dane istniejącego klienta. Procedura automatycznie rozpoznaje typ klienta (firma lub osoba) i aktualizuje odpowiednie tabele podrzędne w ramach jednej transakcji, zachowując spójność danych kontaktowych i identyfikacyjnych.

```sql
CREATE OR ALTER PROCEDURE sp_update_client_data
    @client_id INT,
    @email NVARCHAR(150) = NULL,
    @phone NVARCHAR(12) = NULL,
    @city NVARCHAR(100) = NULL,
    @address NVARCHAR(150) = NULL,
    @city_code NVARCHAR(10) = NULL,

    @company_name NVARCHAR(255) = NULL,
    @nip NVARCHAR(10) = NULL,
    @first_name NVARCHAR(100) = NULL,
    @last_name NVARCHAR(100) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        -- dane wspolne dla typow klientow
        UPDATE clients
        SET email = ISNULL(@email, email),
            phone = ISNULL(@phone, phone),
            city = ISNULL(@city, city),
            address = ISNULL(@address, address),
            city_code = ISNULL(@city_code, city_code)
        WHERE client_id = @client_id;

        -- (jesli klient to firma)
        IF EXISTS (SELECT 1 FROM clients_company WHERE client_id = @client_id)
        BEGIN
            UPDATE clients_company
            SET company_name = ISNULL(@company_name, company_name),
                nip = ISNULL(@nip, nip)
            WHERE client_id = @client_id;
        END
        -- (jesli klient to osoba prywatna)
```

```sql
        ELSE IF EXISTS (SELECT 1 FROM clients_individual WHERE client_id = @client_id)
        BEGIN
            UPDATE clients_individual
            SET first_name = ISNULL(@first_name, first_name),
                last_name = ISNULL(@last_name, last_name)
            WHERE client_id = @client_id;
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.8 sp_add_employee_role

Opis: Dodaje nową rolę (stanowisko) do słownika ról w systemie. Zapobiega duplikatom, sprawdzając, czy rola o takiej samej nazwie już istnieje.

```sql
CREATE PROCEDURE sp_add_employee_role
    @position_name NVARCHAR(150),
    @description NVARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM roles WHERE possition_name = @position_name)
    BEGIN
        RAISERROR('Błąd: Rola o takiej nazwie już istnieje.', 16, 1);
        RETURN;
    END

    INSERT INTO roles (possition_name, description)
    VALUES (@position_name, @description);
END;
```

4.3.9 sp_add_new_employee

Opis: Rejestruje nowego pracownika w bazie danych. Przypisuje mu rolę i ustawia domyślny status aktywności na '1'. Jeśli nie podano daty zatrudnienia, przyjmuje datę bieżącą.

```sql
CREATE PROCEDURE sp_add_new_employee
    @first_name NVARCHAR(100),
    @last_name NVARCHAR(100),
    @base_salary MONEY,
    @role_id INT,
    @supervisor_id INT = NULL,
    @hired_date DATE = NULL,
    @email NVARCHAR(150),
    @phone NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON;
    IF @hired_date IS NULL
        SET @hired_date = CAST(GETDATE() AS DATE);

    -- domyslnie pracownik jest aktywny (is_active = 1)
```

```
    INSERT INTO employees (first_name, last_name, base_salary, is_active, role_id,
    supervisor_id, hired_date, email, phone)
    VALUES (@first_name, @last_name, @base_salary, 1, @role_id, @supervisor_id,
    @hired_date, @email, @phone);
END;
```

4.3.10 sp_update_employee

Opis: Aktualizuje dane pracownika. W przypadku zmiany statusu na nieaktywny (`is_active=0`), procedura wykonuje dodatkowe czyszczenie: usuwa przyszłe zaplanowane urlopy tego pracownika i automatycznie zamyka wszelkie trwające nieobecności z datą dzisiejszą, aby zachować spójność kalendarza pracy.

```
CREATE PROCEDURE sp_update_employee
    @employee_id INT,
    @first_name NVARCHAR(100) = NULL,
    @last_name NVARCHAR(100) = NULL,
    @base_salary MONEY = NULL,
    @is_active BIT = NULL, -- zmiana ktora moze wywolywac dodatkowe operacje!!!
    @role_id INT = NULL,
    @supervisor_id INT = NULL,
    @email NVARCHAR(150) = NULL,
    @phone NVARCHAR(20) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        UPDATE employees
        SET first_name = ISNULL(@first_name, first_name),
            last_name = ISNULL(@last_name, last_name),
            base_salary = ISNULL(@base_salary, base_salary),
            is_active = ISNULL(@is_active, is_active),
            role_id = ISNULL(@role_id, role_id),
            supervisor_id = ISNULL(@supervisor_id, supervisor_id),
            email = ISNULL(@email, email),
            phone = ISNULL(@phone, phone)
        WHERE employee_id = @employee_id;

        -- dezaktywacja pracownika
        IF @is_active = 0
        BEGIN
            -- usuwanie jego przyszlych urlopow
            DELETE FROM employee_time_offs
            WHERE employee_id = @employee_id
              AND start_date > CAST(GETDATE() AS DATE);

            -- zakoncz (na dzis) jego trwajace urlopy
            UPDATE employee_time_offs
            SET end_date = CAST(GETDATE() AS DATE)
            WHERE employee_id = @employee_id
              AND start_date <= CAST(GETDATE() AS DATE)
              AND end_date > CAST(GETDATE() AS DATE);
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
```

```
END;
```

4.3.11 sp_add_employee_time_off

Opis: Rejestruje urlop lub zwolnienie lekarskie. Waliduje poprawność dat (czy koniec nie jest przed początkiem), sprawdza status pracownika (czy jest aktywny) oraz weryfikuje, czy w podanym terminie nie istnieje już inny urlop dla tego pracownika (zapobieganie nakładaniu się dat).

```
CREATE PROCEDURE sp_add_employee_time_off
    @employee_id INT,
    @start_date DATE,
    @end_date DATE,
    @reason NVARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;

    -- warunek integralnosciowy!
    IF @end_date < @start_date
    BEGIN
        RAISERROR('Błąd: Data zakończenia urlopu nie może być wcześniejsza niż data rozpoczęcia.',16,1);
        RETURN;
    END

    -- czy on wg jest aktywny?
    IF EXISTS (SELECT 1 FROM employees WHERE employee_id = @employee_id AND is_active = 0)
    BEGIN
        RAISERROR('Błąd: Nie można dodać urlopu dla nieaktywnego pracownika.',16,1);
        RETURN;
    END

    -- czy urlop sie nie naklada na jakis inny??
    IF EXISTS (
        SELECT 1 FROM employee_time_offs
        WHERE employee_id = @employee_id
          AND (
               (@start_date BETWEEN start_date AND end_date) OR
               (@end_date BETWEEN start_date AND end_date) OR
               (start_date BETWEEN @start_date AND @end_date)
          )
    )
    BEGIN
        RAISERROR('W podanym terminie pracownik ma już zaplanowany inny urlop.',16, 1);
        RETURN;
    END

    INSERT INTO employee_time_offs (employee_id, start_date, end_date, reason)
    VALUES (@employee_id, @start_date, @end_date, @reason);
END;
```

4.3.12 sp_upsert_product

Opis: Zaawansowana procedura zarządzania produktem. Umożliwia dodanie nowego produktu lub edycję istniejącego. Przy dodawaniu nowego produktu automatycznie inicjuje jego stan ma-

gazynowy (`inventory_products`) oraz przypisuje mu aktualną stawkę VAT w tabeli historycznej `products_vat`, dbając o spójność danych finansowych i logistycznych.

```
CREATE PROCEDURE sp_upsert_product
    @product_id INT = NULL,
    @product_name NVARCHAR(255) = NULL,
    @base_price MONEY = NULL,
    @unit_space_needed INT = NULL,
    @category_id INT = NULL,
    @needed_hours INT = NULL,
    @is_active BIT = 1,
    @vat_amount DECIMAL(3,2) = 0.23,
    @base_inventory_min INT = 10,
    @base_inventory_limit INT = 100
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        DECLARE @current_product_id INT = @product_id;

        IF @base_inventory_limit < @base_inventory_min
        BEGIN
            RAISERROR('Błąd: Podaj poprawne wartości limitów magazynowych', 16, 1);
            ROLLBACK; RETURN;
        END

        -- update/insert w tabeli products
        IF @product_id IS NOT NULL AND EXISTS (SELECT 1 FROM products WHERE product_id = @product_id)
        BEGIN
            UPDATE products SET
                product_name = ISNULL(@product_name, product_name),
                base_price = ISNULL(@base_price, base_price),
                unit_space_needed = ISNULL(@unit_space_needed, unit_space_needed),
                category_id = ISNULL(@category_id, category_id),
                needed_hours = ISNULL(@needed_hours, needed_hours),
                is_active = ISNULL(@is_active, is_active)
            WHERE product_id = @product_id;

            DECLARE @current_inventory_min INT, @current_inventory_limit INT;
            SELECT @current_inventory_min = ip.min_units, @current_inventory_limit=ip.max_units FROM inventory_products AS ip
            WHERE ip.product_id=@product_id;

            IF @current_inventory_limit <> @base_inventory_limit
            BEGIN
                UPDATE inventory_products SET max_units=@base_inventory_limit WHERE product_id=@product_id;
            END
            IF @current_inventory_min <> @base_inventory_min
            BEGIN
                UPDATE inventory_products SET min_units=@base_inventory_min WHERE product_id=@product_id;
            END
        END
        ELSE
        BEGIN
            -- warunki integralnosci dla nowego produktu
            IF @product_name IS NULL OR @base_price IS NULL OR @base_price < 0 OR @category_id IS NULL OR @needed_hours IS NULL OR @vat_amount is NULL
            BEGIN
                RAISERROR('Błąd: Podaj poprawną nazwę, cene, kategorię i roboczogodziny dla nowego produktu.', 16, 1);
```

```
                ROLLBACK; RETURN;
            END

            INSERT INTO products (product_name, base_price, unit_space_needed,
    category_id, needed_hours, is_active)
            VALUES (@product_name, @base_price, ISNULL(@unit_space_needed, 1),
    @category_id, @needed_hours, @is_active);

            SET @current_product_id = SCOPE_IDENTITY();

            -- wpis do magazynu dla nowego produktu
            INSERT INTO inventory_products (product_id, available_units, min_units,
    max_units)
            VALUES (@current_product_id, 0, @base_inventory_min, @base_inventory_limit);

            DECLARE @found_vat_id INT;

            -- szukamy id stawki w tabeli vat
            SELECT TOP 1 @found_vat_id = vat_id FROM vat WHERE amount = @vat_amount;

            IF @found_vat_id IS NULL
            BEGIN
                INSERT INTO vat (amount) VALUES (@vat_amount);
                SET @found_vat_id = SCOPE_IDENTITY();
            END

            INSERT INTO products_vat (product_id, vat_id, start_date, end_date)
            VALUES (@current_product_id, @found_vat_id, CAST(GETDATE() AS DATE), NULL);

        END

        COMMIT TRANSACTION;
        SELECT @current_product_id AS product_id;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.13 sp_update_product_vat

Opis: Aktualizuje stawkę VAT dla produktu w sposób historyczny. Zamyka okres obowiązywania dotychczasowej stawki (ustawiając end_date na dziś) i dodaje nowy wpis z nową stawką obowiązującą od dziś. Dzięki temu system zachowuje pełną historię zmian podatkowych.

```
CREATE PROCEDURE sp_update_product_vat
    @product_id INT,
    @new_vat_amount DECIMAL(3,2)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        DECLARE @new_vat_id INT;

        -- znajdz lub dodaj jak trzeba do slownika nowa stawke vat
        SELECT TOP 1 @new_vat_id = vat_id FROM vat WHERE amount = @new_vat_amount;

        IF @new_vat_id IS NULL
        BEGIN
```

```
            INSERT INTO vat (amount) VALUES (@new_vat_amount);
            SET @new_vat_id = SCOPE_IDENTITY();
        END

        -- zakonczenie stawki (poprzedniej) dla tego produktu
        UPDATE products_vat
        SET end_date = CAST(GETDATE() AS DATE)
        WHERE product_id = @product_id
          AND end_date IS NULL
          AND vat_id <> @new_vat_id;

        IF NOT EXISTS (SELECT 1 FROM products_vat WHERE product_id = @product_id AND
    vat_id = @new_vat_id AND end_date IS NULL)
        BEGIN
            INSERT INTO products_vat (product_id, vat_id, start_date, end_date)
            VALUES (@product_id, @new_vat_id, CAST(GETDATE() AS DATE), NULL);
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.14 sp_upsert_product_recipe

Opis: Zarządza składem produktu (BOM). Pozwala dodać nową część do receptury lub zmienić ilość już przypisanej części. Jest to podstawowe narzędzie do definiowania struktury produktu.

```
CREATE PROCEDURE sp_upsert_product_recipe
    @product_id INT,
    @part_id INT,
    @quantity_needed INT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM product_recipes WHERE product_id = @product_id AND part_id =
     @part_id)
    BEGIN
        UPDATE product_recipes
        SET quantity_needed = @quantity_needed
        WHERE product_id = @product_id AND part_id = @part_id;
    END
    ELSE
    BEGIN
        INSERT INTO product_recipes (product_id, part_id, quantity_needed)
        VALUES (@product_id, @part_id, @quantity_needed);
    END
END;
```

4.3.15 sp_add_product_category

Opis: Dodaje nową kategorię produktów do słownika, weryfikując uprzednio unikalność jej nazwy.

```
CREATE PROCEDURE sp_add_product_category
```

```
    @name NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (SELECT 1 FROM categories_for_product WHERE name = @name)
    BEGIN
        RAISERROR('Błąd: Kategoria o tej nazwie już istnieje.', 16, 1);
        RETURN;
    END

    INSERT INTO categories_for_product (name)
    VALUES (@name);
END;
```

4.3.16 sp_upsert_production

Opis: Procedura planowania produkcji. Pozwala utworzyć nowe zlecenie produkcyjne (automatycznie nadając status 'Planned' i przewidywaną datę zakończenia) lub zaktualizować parametry istniejącego, jeszcze nie rozpoczętego zlecenia.

```
CREATE PROCEDURE sp_upsert_production
    @production_id INT = NULL,
    @product_id INT = NULL,
    @quantity INT = NULL,
    @work_place_id INT = NULL,
    @supervisor_id INT = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF @production_id IS NOT NULL AND EXISTS (SELECT 1 FROM productions WHERE
    production_id = @production_id)
    BEGIN
        UPDATE productions SET
            product_id = ISNULL(@product_id, product_id),
            quantity = ISNULL(@quantity, quantity),
            work_place_id = ISNULL(@work_place_id, work_place_id),
            supervisor_id = ISNULL(@supervisor_id, supervisor_id)
        WHERE production_id = @production_id;
    END
    ELSE
    BEGIN
        IF @product_id IS NULL OR @quantity IS NULL OR @supervisor_id IS NULL
        BEGIN
            RAISERROR('Błąd: Nowa produkcja wymaga product_id, quantity i supervisor_id.'
    , 16, 1);
            RETURN;
        END

        INSERT INTO productions (product_id, quantity, work_place_id, supervisor_id)
        VALUES (@product_id, @quantity, @work_place_id, @supervisor_id);

        -- status 1 - planowana
        INSERT INTO production_status (production_id, status_id, start_date,
    planned_end_date)
        VALUES (SCOPE_IDENTITY(), 1, GETDATE(), DATEADD(day, 2, GETDATE()));
    END
END;
```

4.3.17 sp_upsert_employee_to_production

Opis: Przypisuje pracownika do zespołu realizującego konkretne zlecenie produkcyjne. Umożliwia również aktualizację liczby przepracowanych przez niego godzin przy tym zleceniu.

```sql
CREATE PROCEDURE sp_upsert_employee_to_production
    @production_id INT,
    @employee_id INT,
    @hours_logged INT = 0
AS
BEGIN
    IF EXISTS (SELECT 1 FROM production_team_assignments WHERE production_id =
    @production_id AND employee_id = @employee_id)
    BEGIN
        UPDATE production_team_assignments SET hours_logged = @hours_logged
        WHERE production_id = @production_id AND employee_id = @employee_id;
    END
    ELSE
    BEGIN
        INSERT INTO production_team_assignments (production_id, employee_id, hours_logged
    )
        VALUES (@production_id, @employee_id, @hours_logged);
    END
END;
```

4.3.18 sp_remove_employee_from_production

Opis: Usuwa pracownika z przypisania do danego zlecenia produkcyjnego.

```sql
CREATE PROCEDURE sp_remove_employee_from_production
    @production_id INT,
    @employee_id INT
AS
BEGIN
    DELETE FROM production_team_assignments
    WHERE production_id = @production_id AND employee_id = @employee_id;
END;
```

4.3.19 sp_replace_production_employee

Opis: Umożliwia szybką zamianę pracownika przy zleceniu (np. w przypadku choroby). Procedura usuwa starego pracownika z zespołu i dodaje na jego miejsce nowego, przenosząc dotychczas zaraportowane godziny (lub zerując je w zależności od logiki biznesowej - tutaj zerujemy dla nowego). Weryfikuje, czy pracownik zastępowany faktycznie był przypisany do zlecenia.

```sql
CREATE PROCEDURE sp_replace_production_employee
    @production_id INT,
    @old_employee_id INT,
    @new_employee_id INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        IF NOT EXISTS (
            SELECT 1
            FROM production_team_assignments
            WHERE production_id = @production_id AND employee_id = @old_employee_id
        )
```

```sql
        BEGIN
            RAISERROR('Błąd: Wskazany pracownik o id: %d nie jest przypisany do produkcji o id: %d.', 16, 1, @old_employee_id, @production_id);
            ROLLBACK; RETURN;
        END

        -- czy nowy pracownik nie jest juz tam przypisany!!
        IF EXISTS (
            SELECT 1
            FROM production_team_assignments
            WHERE production_id = @production_id AND employee_id = @new_employee_id
        )
        BEGIN
            RAISERROR('Błąd: Nowy pracownik o id: %d jest już przypisany do tej produkcji.', 16, 1, @new_employee_id);
            ROLLBACK; RETURN;
        END

        -- podmianka -> usuniecie starego -> dodanie nowego
        INSERT INTO production_team_assignments (production_id, employee_id, hours_logged)
        SELECT @production_id, @new_employee_id, hours_logged
        FROM production_team_assignments
        WHERE production_id = @production_id AND employee_id = @old_employee_id;

        DELETE FROM production_team_assignments
        WHERE production_id = @production_id AND employee_id = @old_employee_id;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.20 sp_start_production

Opis: Kluczowa procedura uruchamiająca produkcję. Sprawdza, czy w magazynie dostępna jest wystarczająca ilość wszystkich części zdefiniowanych w recepturze produktu. Jeśli tak, **zdejmuje surowce ze stanu magazynowego** i zmienia status produkcji na 'In Progress' (ID 2). Jeśli brakuje części, blokuje uruchomienie i zwraca błąd. Oblicza również szacowaną datę zakończenia.

```sql
CREATE OR ALTER PROCEDURE sp_start_production
    @production_id INT,
    @planned_end_date DATETIME = NULL
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRANSACTION;
    BEGIN TRY
        -- produkcja nie zostala juz rozpoczeta lub zakonczona?
        IF EXISTS (SELECT 1 FROM production_status WHERE production_id = @production_id AND status_id >= 2 AND end_date IS NULL)
        BEGIN
            RAISERROR('Błąd: Produkcja jest już w toku lub została zakończona.', 16, 1);
            ROLLBACK; RETURN;
        END

```

```sql
        -- sprawdzenie dostepnosci wymaganych czesci
        IF EXISTS (
            SELECT 1
            FROM product_recipes pr
            JOIN productions p ON p.product_id = pr.product_id
            JOIN inventory_parts ip ON ip.part_id = pr.part_id
            WHERE p.production_id = @production_id
              AND ip.available_units < (pr.quantity_needed * p.quantity)
        )
        BEGIN
            RAISERROR('Błąd: Niewystarczająca ilość części w magazynie.', 16, 1);
            ROLLBACK; RETURN;
        END

        -- logika planned_end_date
        IF @planned_end_date IS NULL
        BEGIN
            DECLARE @days_needed INT;

            SELECT @days_needed = CEILING((CAST(p.quantity AS FLOAT) * ISNULL(prod.needed_hours, 1)) / 24.0)
            FROM productions p
            JOIN products prod ON p.product_id = prod.product_id
            WHERE p.production_id = @production_id;

            SET @planned_end_date = DATEADD(day, ISNULL(@days_needed, 1), GETDATE());
        END
        ELSE
        BEGIN
            -- walidacja podanej daty: czy planowany koniec nie jest w przeszlosci?
            IF @planned_end_date <= GETDATE()
            BEGIN
                RAISERROR('Błąd: Planowana data zakończenia nie może być wcześniejsza niż obecny czas.', 16, 1);
                ROLLBACK; RETURN;
            END
        END

        -- odjecie czesci z magazynu
        UPDATE ip
        SET ip.available_units = ip.available_units - (pr.quantity_needed * p.quantity)
        FROM inventory_parts ip
        JOIN product_recipes pr ON ip.part_id = pr.part_id
        JOIN productions p ON p.product_id = pr.product_id
        WHERE p.production_id = @production_id;

        -- zakonczenie poprzedniego stanu (powinien  byc np. wstrzymany lub planowany)
        UPDATE production_status
        SET end_date = GETDATE()
        WHERE production_id = @production_id AND end_date IS NULL;

        -- dodanie nowego statusu In Production (ID 2)
        INSERT INTO production_status (production_id, status_id, start_date, planned_end_date)
        VALUES (
            @production_id,
            2,
            GETDATE(),
            @planned_end_date
        );

        COMMIT TRANSACTION;
    END TRY
```

```
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.21 sp_finish_production

Opis: Procedura finalizująca proces produkcyjny. Zmienia status zlecenia na 'Completed' (lub inny końcowy, np. 'Cancelled'). Jeśli produkcja zakończyła się sukcesem, **dodaje wyprodukowane wyroby do magazynu produktów gotowych** i zwalnia wszystkich pracowników przypisanych do tego zlecenia.

```
CREATE PROCEDURE sp_finish_production
    @production_id INT,
    @final_status_id INT = 5 -- ID statusu "Completed"
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        -- statusy koncowe
        IF @final_status_id NOT BETWEEN 5 AND 6
        BEGIN
            RAISERROR('Błąd: Podany status nie jest końcowy.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        IF EXISTS (
            SELECT 1
            FROM production_status
            WHERE production_id = @production_id
              AND status_id BETWEEN 5 AND 6
        )
        BEGIN
            RAISERROR('Błąd: Ta produkcja została już wcześniej zakończona lub anulowana.
    ', 16, 1);
            ROLLBACK TRANSACTION; RETURN;
        END

        IF @final_status_id=5
        BEGIN
            UPDATE ip
            SET ip.available_units = ip.available_units + p.quantity
            FROM inventory_products ip
            JOIN productions p ON p.product_id = ip.product_id
            WHERE p.production_id = @production_id;
        END

        -- update trwajacych statusow
        UPDATE production_status SET end_date = GETDATE() WHERE production_id =
    @production_id AND end_date IS NULL;

        INSERT INTO production_status (production_id, status_id, start_date,
    planned_end_date, end_date)
        VALUES (@production_id, @final_status_id, GETDATE(), GETDATE(), GETDATE());

        -- usuniecie przydzialow pracownikow
        DELETE FROM production_team_assignments WHERE production_id = @production_id;

```

```
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.22 sp_add_work_place

Opis: Dodaje nowe stanowisko pracy (gniazdo produkcyjne) do systemu.

```
CREATE PROCEDURE sp_add_work_place
    @description NVARCHAR(255)
AS
BEGIN
    INSERT INTO work_places (work_place_description) VALUES (@description);
END;
```

4.3.23 sp_confirm_order

Opis: Zatwierdza zamówienie klienta do realizacji. Sprawdza dostępność produktów w magazynie. Jeśli towar jest dostępny, **rezerwuje go (zdejmuje ze stanu)** i zmienia status zamówienia na 'Ready for Shipment' (ID 2). Jeśli brakuje towaru, operacja jest wycofywana.

```
CREATE PROCEDURE sp_confirm_order
    @order_id INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM order_details WHERE order_id = @order_id)
        BEGIN
            RAISERROR('Błąd: Nie można zatwierdzić pustego zamówienia.', 16, 1);
            ROLLBACK; RETURN;
        END

        -- odejmij ze stanu - safe bo transakcja!!
        UPDATE ip
        SET ip.available_units = ip.available_units - od.quantity
        FROM inventory_products ip
        JOIN order_details od ON ip.product_id = od.product_id
        WHERE od.order_id = @order_id;

        -- czy mamy tyle na stanie??
        IF EXISTS (SELECT 1 FROM inventory_products WHERE available_units < 0)
        BEGIN
            RAISERROR('Błąd: Niewystarczająca ilość produktu w magazynie.', 16, 1);
            ROLLBACK; RETURN;
        END

        -- status_for_order o id 2 - Ready for Shipment
        INSERT INTO order_status (order_id, status_id, start_date, planned_end_date,
    end_date)
        VALUES (@order_id, 2, GETDATE(), GETDATE(), GETDATE());

        COMMIT TRANSACTION;
    END TRY
```

```
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.24 sp_upsert_order

Opis: Tworzy nowe zamówienie lub aktualizuje nagłówek istniejącego zamówienia (np. zmiana adresu dostawy, spedytora). Zwraca ID nowego zamówienia w przypadku wstawiania.

```
CREATE PROCEDURE sp_upsert_order
    @order_id INT = NULL,
    @shipper_id INT = NULL,
    @delivery_city NVARCHAR(100) = NULL,
    @delivery_address NVARCHAR(150) = NULL,
    @delivery_city_code NVARCHAR(10) = NULL,
    @supervisor_id INT = NULL,
    @order_discount DECIMAL(3,2) = NULL,
    @client_id INT = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF @order_id IS NOT NULL AND EXISTS (SELECT 1 FROM orders WHERE order_id = @order_id)
    BEGIN
        UPDATE orders SET
            shipper_id = ISNULL(@shipper_id, shipper_id),
            delivery_city = ISNULL(@delivery_city, delivery_city),
            delivery_address = ISNULL(@delivery_address, delivery_address),
            delivery_city_code = ISNULL(@delivery_city_code, delivery_city_code),
            supervisor_id = ISNULL(@supervisor_id, supervisor_id),
            order_discount = ISNULL(@order_discount, order_discount),
            client_id = ISNULL(@client_id, client_id)
        WHERE order_id = @order_id;
    END
    ELSE
    BEGIN
        IF @shipper_id IS NULL OR @client_id IS NULL OR @supervisor_id IS NULL
        BEGIN
            RAISERROR('Błąd: Nowe zamówienie wymaga shipper_id, client_id i supervisor_id
    .', 16, 1);
            RETURN;
        END
        INSERT INTO orders (shipper_id, order_date, delivery_city, delivery_address,
    supervisor_id, order_discount, client_id, delivery_city_code)
        VALUES (@shipper_id, GETDATE(), @delivery_city, @delivery_address, @supervisor_id
    , ISNULL(@order_discount, 0), @client_id, @delivery_city_code);

        SELECT SCOPE_IDENTITY() AS NewOrderID;
    END
END;
```

4.3.25 sp_add_order_detail_with_overflow

Opis: Procedura inteligentnego dodawania pozycji do zamówienia z funkcją ”overflow”. Sprawdza limit logistyczny (np. 90% pojemności magazynowej przeznaczonej na dany produkt). Jeśli zamówiona ilość przekracza dostępny limit, procedura: 1. Wypełnia bieżące zamówienie do dostępnego limitu. 2. Automatycznie tworzy nowe zamówienie (klonując dane nagłówkowe) dla

nadmiarowej ilości. Funkcja ta jest rekurencyjna, co pozwala na obsługę bardzo dużych zamówień dzielonych na wiele mniejszych.

```sql
CREATE OR ALTER PROCEDURE sp_add_order_detail_with_overflow
    @order_id INT,
    @product_id INT,
    @quantity INT,
    @unit_price MONEY = NULL
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;
    BEGIN TRY
        -- dane o cenie i o stanie magazynowym produktu
        IF @unit_price IS NULL
            SELECT @unit_price = base_price FROM products WHERE product_id = @product_id;

        DECLARE @max_units INT, @current_stock INT;
        SELECT @max_units = max_units, @current_stock = available_units
        FROM inventory_products WHERE product_id = @product_id;

        -- limit dla zamowienia - 90% przestrzeni magazynowej
        DECLARE @storage_limit INT = @max_units * 0.9;
        DECLARE @available_space INT = @storage_limit - @current_stock;

        IF @available_space < 0 SET @available_space = 0;

        DECLARE @to_insert_now INT, @overflow_qty INT;

        IF @quantity <= @available_space
        BEGIN
            SET @to_insert_now = @quantity;
            SET @overflow_qty = 0;
        END
        ELSE
        BEGIN
            SET @to_insert_now = @available_space;
            SET @overflow_qty = @quantity - @available_space;
        END

        -- insert lub update w biezacym zamowieniu
        IF @to_insert_now > 0
        BEGIN
            IF EXISTS (SELECT 1 FROM order_details WHERE order_id = @order_id AND
    product_id = @product_id)
            BEGIN
                UPDATE order_details SET quantity = quantity + @to_insert_now
                WHERE order_id = @order_id AND product_id = @product_id;
            END
            ELSE
            BEGIN
                INSERT INTO order_details (order_id, product_id, quantity, unit_price)
                VALUES (@order_id, @product_id, @to_insert_now, @unit_price);
            END
        END

        -- nadmiar! - do nowego zamowienia
        IF @overflow_qty > 0
        BEGIN
            DECLARE @new_order_id INT;
            -- dane zamowienia sa takie same!
            INSERT INTO orders (shipper_id, order_date, delivery_city, delivery_address,
    supervisor_id, order_discount, client_id, delivery_city_code)
```

```
            SELECT shipper_id, GETDATE(), delivery_city, delivery_address, supervisor_id, order_discount, client_id, delivery_city_code
            FROM orders WHERE order_id = @order_id;

            SET @new_order_id = SCOPE_IDENTITY();

            -- rekurencyjny rozklad
            EXEC sp_add_order_detail_with_overflow @new_order_id, @product_id, @overflow_qty, @unit_price;
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

4.3.26 sp_delete_order_detail

Opis: Usuwa pozycję (produkt) z zamówienia.

```
CREATE PROCEDURE sp_delete_order_detail
    @order_id INT,
    @product_id INT
AS
BEGIN
    DELETE FROM order_details WHERE order_id = @order_id AND product_id = @product_id;
END;
```

4.3.27 sp_upsert_shipper

Opis: Zarządza bazą spedytorów. Aktualizuje dane istniejącego przewoźnika lub dodaje nowego.

```
CREATE PROCEDURE sp_upsert_shipper
    @shipper_id INT = NULL,
    @company_name NVARCHAR(255) = NULL,
    @email VARCHAR(150) = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM shippers WHERE shipper_id = @shipper_id OR email = @email)
    BEGIN
        UPDATE shippers
        SET company_name = ISNULL(@company_name, company_name),
            email = ISNULL(@email, email)
        WHERE shipper_id = @shipper_id OR email = @email;
    END
    ELSE
    BEGIN
        IF @company_name IS NULL OR @email IS NULL
        BEGIN
            RAISERROR('Błąd: Przy tworzeniu nowego przewoźnika podaj nazwę i email.', 16, 1);
            RETURN;
        END
        INSERT INTO shippers (company_name, email) VALUES (@company_name, @email);
    END
END;
```

4.3.28 sp_upsert_payment

Opis: Rejestruje płatność za zamówienie. Może służyć do dodania nowej płatności lub korekty kwoty istniejącej.

```sql
CREATE PROCEDURE sp_upsert_payment
    @payment_id INT = NULL,
    @order_id INT = NULL,
    @amount MONEY = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF @payment_id IS NOT NULL AND EXISTS (SELECT 1 FROM payments WHERE payment_id =
    @payment_id)
    BEGIN
        UPDATE payments SET
            amount = ISNULL(@amount, amount),
            payment_date = GETDATE()
        WHERE payment_id = @payment_id;
    END
    ELSE
    BEGIN
        IF @order_id IS NULL OR @amount IS NULL
        BEGIN
            RAISERROR('Błąd: Nowa płatność wymaga order_id i kwoty.', 16, 1);
            RETURN;
        END
        INSERT INTO payments (order_id, amount, payment_date)
        VALUES (@order_id, @amount, GETDATE());
    END
END;
```

4.3.29 sp_manage_order_status

Opis: Zarządza cyklem życia zamówienia poprzez zmianę jego statusów. Dba o ciągłość historii: zamyka poprzedni aktywny status (ustawia datę końca) i otwiera nowy. Posiada walidację chronologiczną, uniemożliwiając wstawienie statusu z datą, która koliduje z innym aktywnym okresem dla tego samego zamówienia.

```sql
CREATE PROCEDURE sp_manage_order_status
    @order_id INT,
    @status_id INT = NULL,
    @status_name NVARCHAR(30) = NULL,
    @start_date DATETIME = NULL,
    @new_end_date DATETIME = NULL,
    @planned_end_date DATETIME = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- pobieranie status_id po nazwie, jesli id nie zostalo podane
    IF @status_id IS NULL AND @status_name IS NOT NULL
    BEGIN
        SELECT @status_id = status_id FROM status_for_order WHERE status_name =
    @status_name;
    END

    -- mamy stany od 1 do 6 ale do 2 mozemy tylko procedura zatwierdzajaca
    IF @status_id IS NULL OR @status_id NOT BETWEEN 1 AND 6 OR @status_id = 2
    BEGIN
```

```sql
        RAISERROR('Błąd: Nieprawidłowy status. Dopuszczalne ID to 1, 3, 4, 5, 6.', 16, 1);
        RETURN;
    END

    -- (Jesli podano @start_date, zamykamy istniejacy wpis)
    IF @start_date IS NOT NULL
    BEGIN
        UPDATE order_status
        SET end_date = ISNULL(@new_end_date, GETDATE())
        WHERE order_id = @order_id
          AND status_id = @status_id
          AND start_date = @start_date;

        IF @@ROWCOUNT = 0
            RAISERROR('Błąd: Nie znaleziono dokładnie takiego wpisu do aktualizacji.', 16, 1);

        RETURN;
    END

    -- nowy wpis + walidacja nakladania sie terminow
    DECLARE @now DATETIME = GETDATE();

    IF EXISTS (
        SELECT 1 FROM order_status
        WHERE order_id = @order_id
          AND status_id = @status_id
          AND @now BETWEEN start_date AND planned_end_date
          AND end_date IS NULL
    )
    BEGIN
        RAISERROR('Błąd: Nowa data startowa koliduje z istniejacym, aktywnym wpisem o tym samym statusie.', 16, 1);
        RETURN;
    END

    -- zakonczenie poprzednich statusow (end_date IS NULL)
    UPDATE order_status
    SET end_date = @now
    WHERE order_id = @order_id AND end_date IS NULL;

    -- nowy status
    INSERT INTO order_status (order_id, status_id, start_date, planned_end_date)
    VALUES (@order_id, @status_id, @now, ISNULL(@planned_end_date, DATEADD(day, 3, @now)));

END;
```

5 Informacje odnośnie wygenerowanych danych

5.1 Proces generowania danych

Dane testowe zostały wygenerowane przy użyciu modelu językowego ChatGPT. Zastosowano następujący prompt, aby zapewnić spójność i realizm danych:

> ”Wygeneruj zestaw testowy w języku polskim i pliki w formacie CSV: clients.csv (100), clients_company.csv (33), clients_individual.csv (67), products.csv (20), parts.csv (70), product_recipes.csv (161), inventory_parts.csv (70), orders.csv (100), order_details.csv (+-289), productions.csv (98), production_team_assignments.csv (15), payments.csv (50), suppliers.csv (5), supplier_offers.csv (75), shippers.csv (10), vat.csv (5). Daty między 2023-01-01 a 2025-12-31. Zachowaj spójność FK==PK. Format dat YYYY-MM-DD, kody pocztowe dd-ddd, telefony 48XXXXXXXXX. Dane te maja pasować do poniższego schematu bazy. Zadbaj o to by nazwy części, firm, produktów itp. nie były generyczne (np. część1, część2 itd.), oraz aby adresy mailowe nie zawierały polskich znaków.”
> // tutaj został wklejony kod sql generujący strukturę bazy (składał się z poszczególnych definicji, podanych w punkcie 1 i 2)

Użyto narzędzia: ChatGPT (z opcją ”Myślący”).

5.2 Kluczowe słowniki systemowe

Poniższe dane słownikowe (INSERTy) zostały przygotowane ręcznie i stanowią podstawę logiki biznesowej (nie były generowane losowo).

Statusy Produkcji:

```sql
INSERT INTO status_for_production (status_name)
VALUES
    ('Planned'),
    ('In Progress'),
    ('Quality Check'),
    ('Suspended'),
    ('Completed'),
    ('Cancelled');
```

Statusy Zamówień:

```sql
INSERT INTO status_for_order (status_name)
VALUES
    ('In Production'),
    ('Ready for Shipment'),
    ('Return'),
    ('Cancelled'),
    ('Completed'),
    ('Suspended');
```

Role Pracowników:

```sql
INSERT INTO roles (possition_name, description)
VALUES
    ('Production Manager', 'Oversees manufacturing process, production planning, and workflow coordination'),
    ('Production Worker', 'Performs manufacturing tasks, assembles furniture components'),
    ('Quality Control Specialist', 'Inspects finished products, ensures quality standards are met'),
    ('Warehouse Manager', 'Manages inventory, storage, and material flow in warehouse'),
```

```
    ('Warehouse Worker', 'Handles materials, picks orders, manages stock levels'),
    ('Sales Representative', 'Handles customer orders, provides product information, processes sales'),
    ('Sales Manager', 'Leads sales team, develops sales strategies, monitors performance'),
    ('Customer Service Specialist', 'Handles customer inquiries, complaints, and after-sales support'),
    ('Accountant', 'Manages financial records, invoicing, VAT, and cost calculations'),
    ('Procurement Specialist', 'Sources and purchases raw materials, manages supplier relationships'),
    ('Product Designer', 'Designs new furniture models, creates technical specifications'),
    ('IT Support', 'Maintains IT systems, database management, software support'),
    ('CEO', 'Overall company management, strategic decision making'),
    ('HR Specialist', 'Handles recruitment, payroll, employee relations'),
    ('Logistics Coordinator', 'Coordinates shipping, delivery schedules, and transportation');
```

6 Logika Biznesowa: Przepływ Zamowienia przez Magazyn

System implementuje logikę przechodzenia wszystkich zamówień przez magazyn, tj. zamówienie zawsze trafia do magazynu, a dopiero z niego realizowane jest do klienta.

1. Zlozenie zamowienia
Klient sklada zamowienie, co skutkuje dodaniem rekordu do tabeli `orders` oraz pozycji do `order_details`. Zamowienie otrzymuje status poczatkowy 'Planned'.

2. Weryfikacja dostepnosci
Podczas proby zatwierdzenia zamowienia (wywolanie procedury `sp_confirm_order`), system sprawdza stan w tabeli `inventory_products`.

3. Rezerwacja towaru
Jesli w magazynie znajduje sie wystarczajaca ilosc towaru, system natychmiast zdejmuje (odejmuje) wymagana ilosc sztuk z kolumny `available_units`. Operacja ta odbywa sie w ramach transakcji.

4. Zmiana statusu
Po pomyslnym zarezerwowaniu towaru, status zamowienia automatycznie zmienia sie na 'Ready for Shipment'.

5. Obsluga braku towaru
Jesli towaru brakuje, procedura zwraca blad (`RAISERROR`) i blokuje zmiane statusu. Jest to sygnal dla dzialu produkcji (uruchamiany jest trigger `trg_auto_plan_production`) lub dzialu zaopatrzenia.

7 Określenie uprawnień do danych (Role)

W systemie zdefiniowano następujące role bazodanowe:

1. **Role_Manager**: Pełny dostęp do raportów (SELECT na widokach analitycznych).

2. **Role_Sales**: Obsługa zamówień (`INSERT/UPDATE` na `orders`, `clients`).
3. **Role_Warehouse**: Zarządzanie magazynem (UPDATE na `inventory_*`).
4. **Role_Production**: Zarządzanie produkcją (EXEC na procedurach produkcyjnych).

8 Indeksy

W bazie danych zastosowano dwa rodzaje indeksów: indeksy unikalne (zapewniające integralność danych i brak duplikatów w słownikach) oraz indeksy optymalizacyjne (przyspieszające najczęstsze operacje łączenia tabel i filtrowania).

8.1 Wykaz indeksów

Ponizszy listing zawiera pelny kod SQL definiujacy wszystkie indeksy w systemie:

```sql
-- ==============================================
-- 1. INDEKSY UNIKALNE (Integralnośćć danych)
-- ==============================================

-- Słowniki kategorii
CREATE UNIQUE INDEX categories_for_part_idx_1 ON categories_for_part (name ASC);
CREATE UNIQUE INDEX categories_for_product_idx_1 ON categories_for_product (name ASC);

-- Klienci i Firmy (unikalne dane kontaktowe/identyfikacyjne)
CREATE UNIQUE INDEX clients_idx_1 ON clients (phone ASC, email ASC);
CREATE UNIQUE INDEX clients_company_idx_1 ON clients_company (company_name ASC, nip ASC);

-- Pracownicy i Role
CREATE UNIQUE INDEX employees_idx_1 ON employees (email ASC, phone ASC);
CREATE UNIQUE INDEX roles_idx_1 ON roles (possition_name ASC);

-- Dostawcy i Przewoznicy
CREATE UNIQUE INDEX shippers_idx_1 ON shippers (company_name ASC);
CREATE UNIQUE INDEX suppliers_idx_1 ON suppliers (company_name ASC, email ASC);

-- Slowniki statusow
CREATE UNIQUE INDEX status_for_order_idx_1 ON status_for_order (status_name ASC);
CREATE UNIQUE INDEX status_for_production_idx_1 ON status_for_production (status_name ASC
    );

-- ==============================================
-- 2. INDEKSY WYDAJNOSCIOWE (Optymalizacja)
-- ==============================================

CREATE INDEX IX_ProductRecipes_ProductId ON product_recipes(product_id);
-- Przyspiesza wyszukiwanie receptury dla danego produktu (JOIN products-recipes)

CREATE INDEX IX_Orders_OrderDate ON orders(order_date) INCLUDE(client_id, order_discount)
    ;
-- Optymalizuje raportowanie sprzedazy wg dat (przyspiesza widoki statystyczne)

CREATE INDEX IX_ProductionStatus_StartDate ON production_status(start_date) INCLUDE(
    production_id);
-- Przyspiesza filtrowanie produkcji po dacie rozpoczecia (np. w kalendarzu)
```

Listing 31: Definicje indeksów unikalnych i optymalizacyjnych